



香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC 3310

Digital Fundamentals and System Design

Wenchao Miao

Outline Lecture 9 Finite State Machines Registers



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

1 Learning Outcomes

2 Registers

3 Shift Register Operations

4 Shift Register Counters

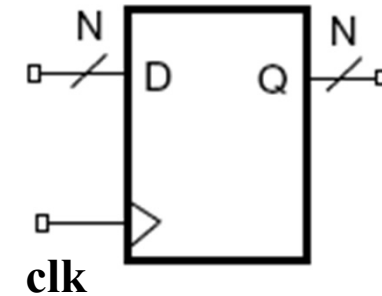
5 Shift Register Applications

1. Learning Outcomes

- 1. Explain how serial in/serial out (SISO), serial in/parallel out (SIPO), parallel in/serial out (PISO), and parallel in/parallel out (PIPO) shift registers operate**
- 2. Describe how a bidirectional shift register operates**
- 3. Determine the sequence of a Johnson counter**
- 4. Set up a ring counter to produce a specified sequence**
- 5. Construct a ring counter from a shift register**
- 6. Use a shift register as a time-delay device**
- 7. Use a shift register to implement a serial-to-parallel data converter**
- 8. Use shift registers to implement serial data communication**
- 9. Use shift registers to build a serial adder**

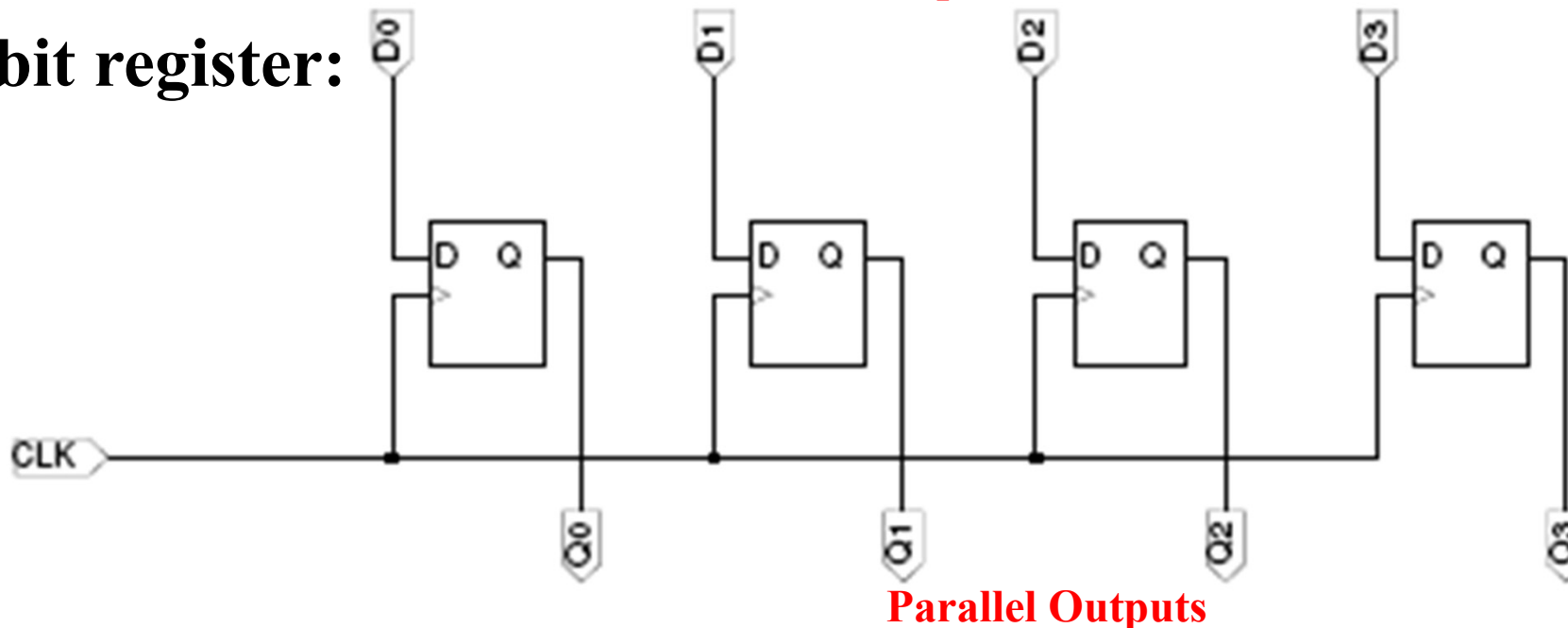
Registers = Memory

- Registers are groups of flip-flops
- A register is a digital circuit with two basic functions:
data storage and **data movement**.
- Basic parallel storage register:
 - Store N-bit information
 - Each FF stores one-bit



Register symbol

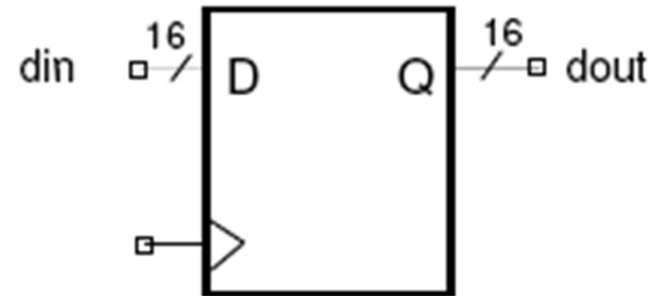
A 4-bit register:



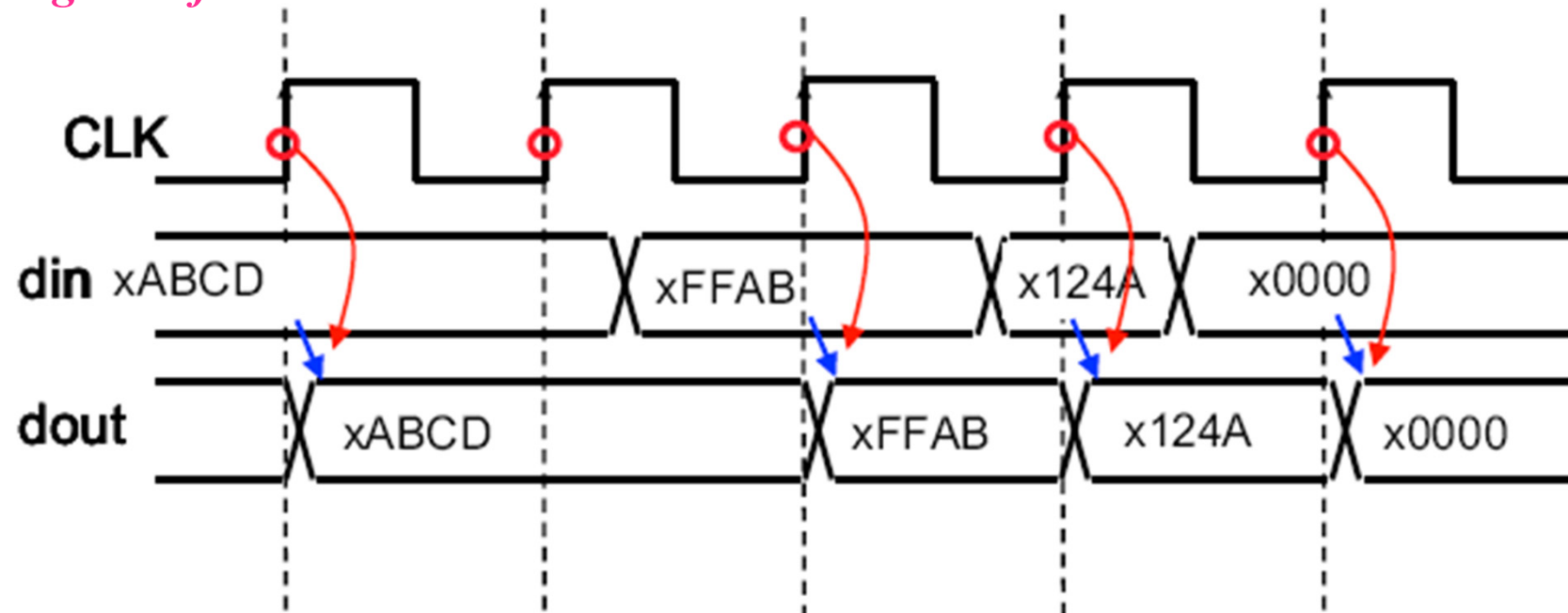
Registers – Timing Waveform



Logic Symbol



Timing waveform



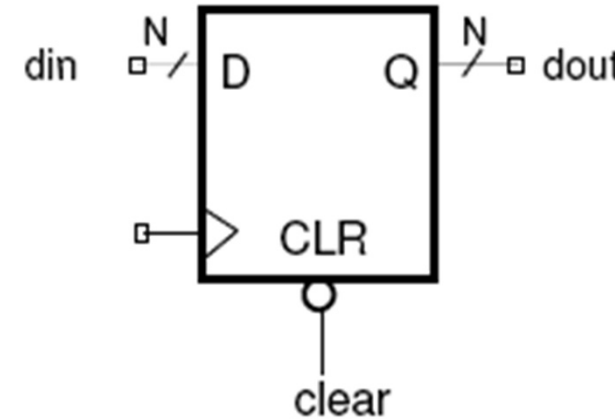
Registers with Asynchronous Clear



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

- Just like FFs, registers can have asynchronous inputs:
 - CLR – resets the outputs to 0
 - PRESET – Set the outputs to 1
- Could be active LOW or active HIGH
 - Active LOW: resets the output to 0 when CLR = 0
 - Active HIGH: resets the output to 0 when CLR = 1
 - Similar for PRESET input
- Asynchronous inputs – change the register output immediately, do not need to wait for the clock edge.

Logic Symbol



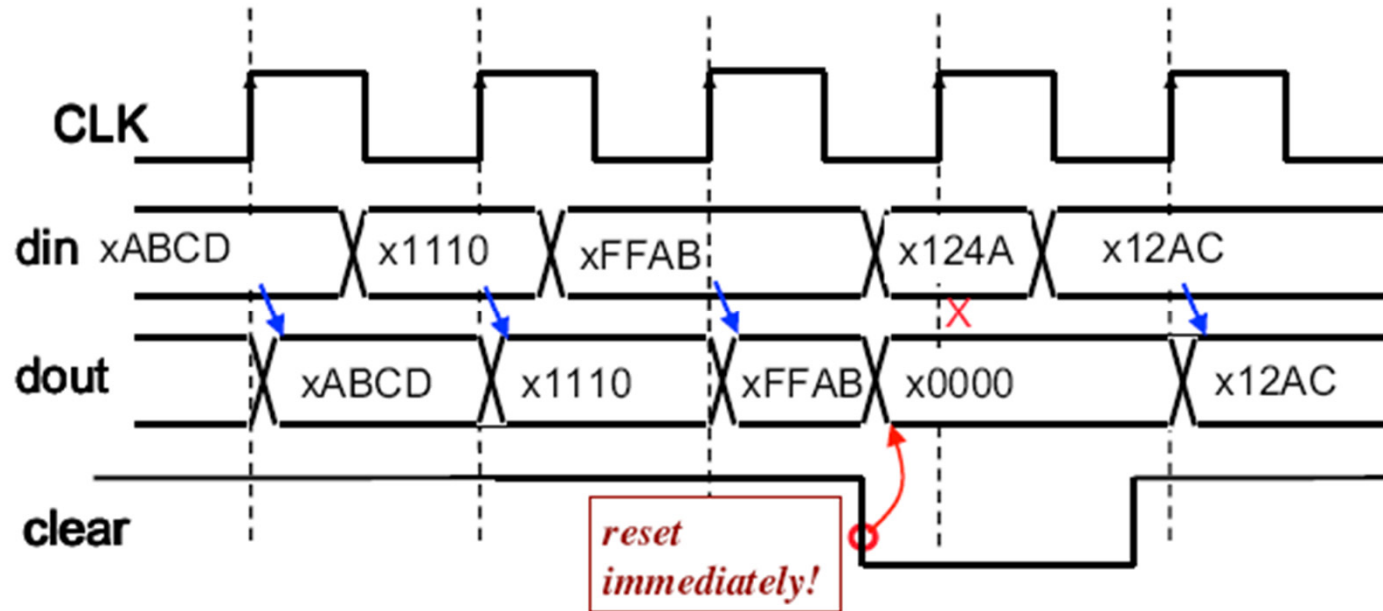
(Active LOW CLR)

4-bit Register with Asynchronous Clear

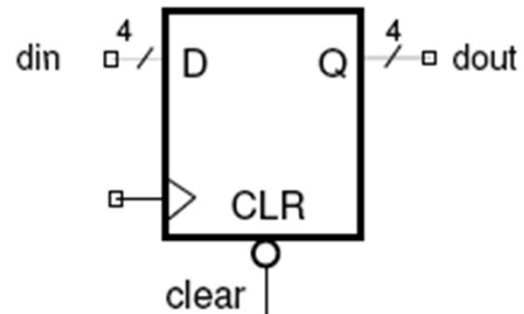


THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

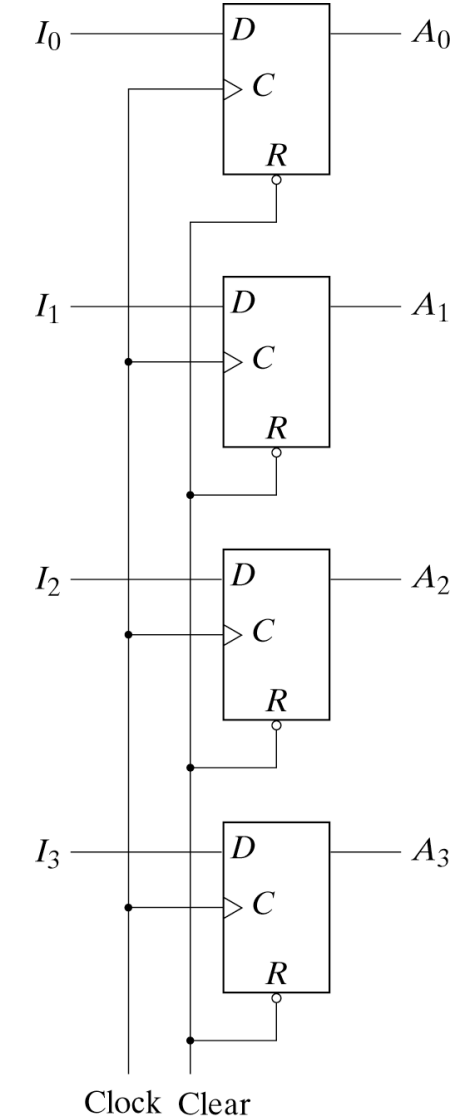
Timing waveform



Logic Symbol

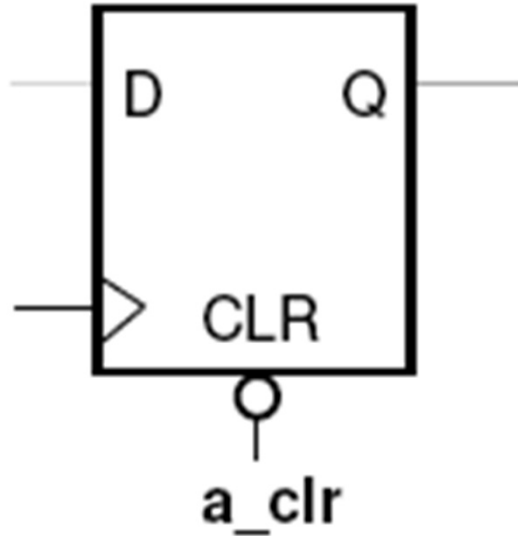


Implementation:

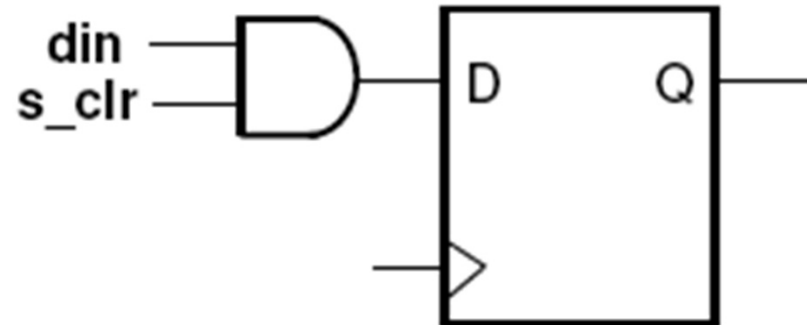


Synchronous vs Asynchronous CLR

- **Asynchronous clear has an immediate effect, do not need to wait for the clock**



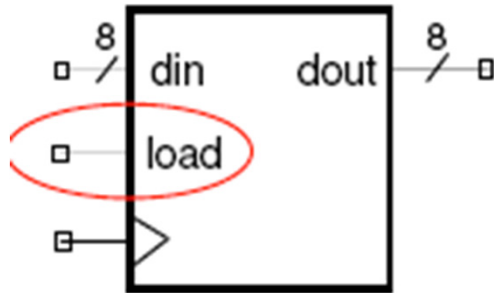
- **Synchronous clear changes the output after the next clock edge, basically, the flip flops just load a “ZERO” to the output at the next clock edge**
 - This example show active LOW clear signal.
Active HIGH can also be used. How can you do it?



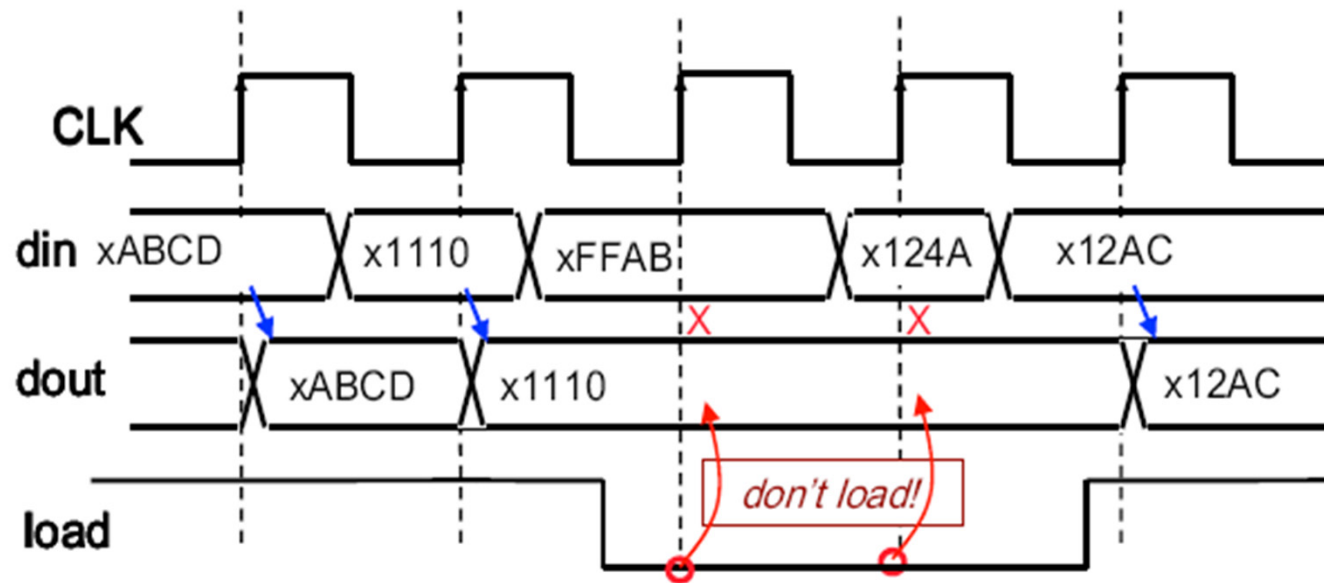
Registers with Parallel Load

- **LOAD (or ENABLE)** input to control whether the data is to be loaded into the register in the next clock edge
- For active HIGH, if load (enable) = 1, the input data will be loaded and stored in the register at the coming clock edge, otherwise no change at the output at the clock edge.
- Load is a synchronous input, i.e. effect at the output only happen at the next clock edge.

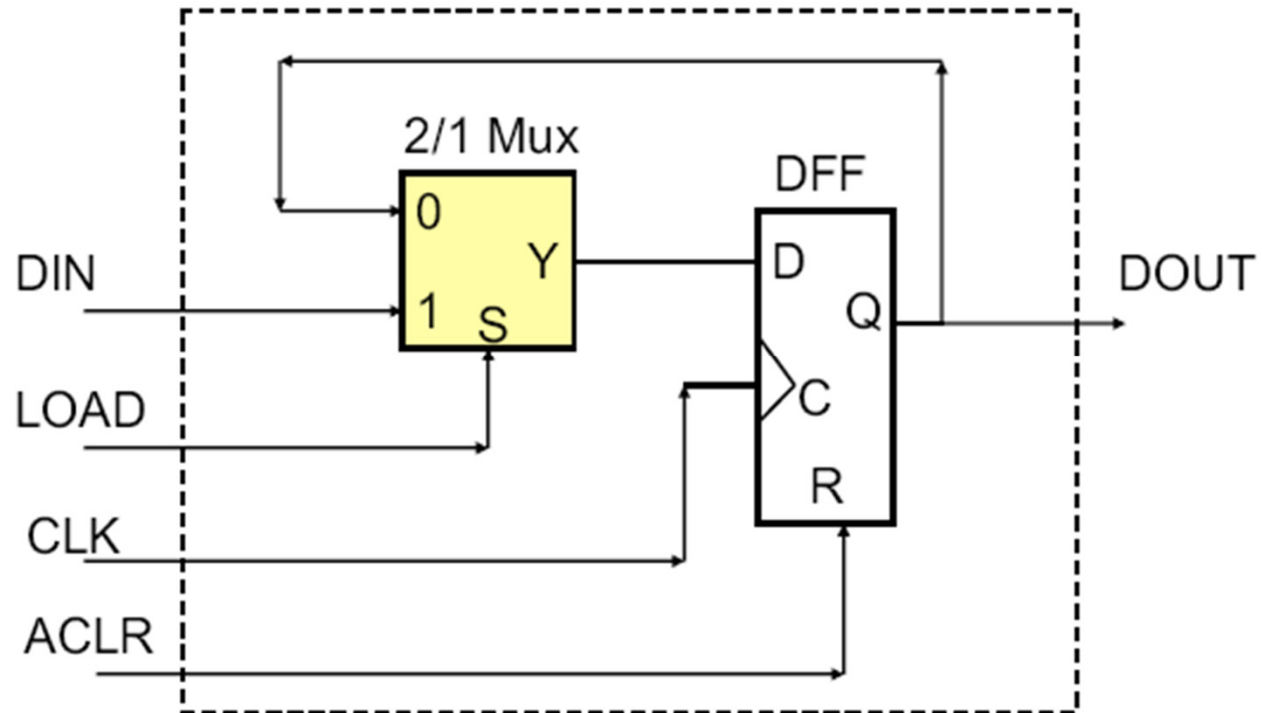
Logic Symbol



Timing waveform

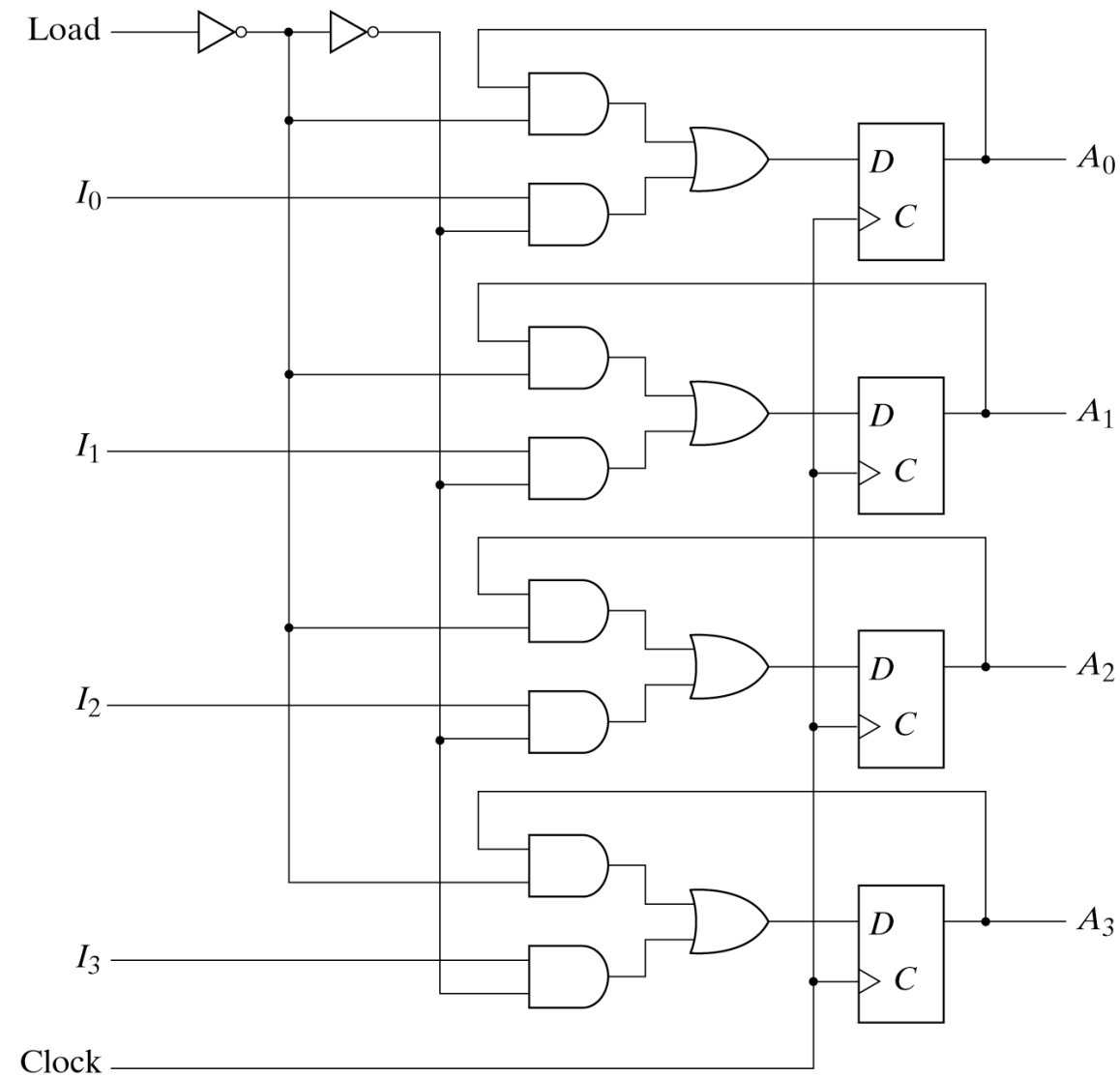


Implementation of 1-bit “Register with Load” using D-Flip Flop and Mux



Note that DFF simply loads old value when $\text{LOAD} = 0$. DFF is loaded every clock cycle.

4-bit Register with Load (Enable)

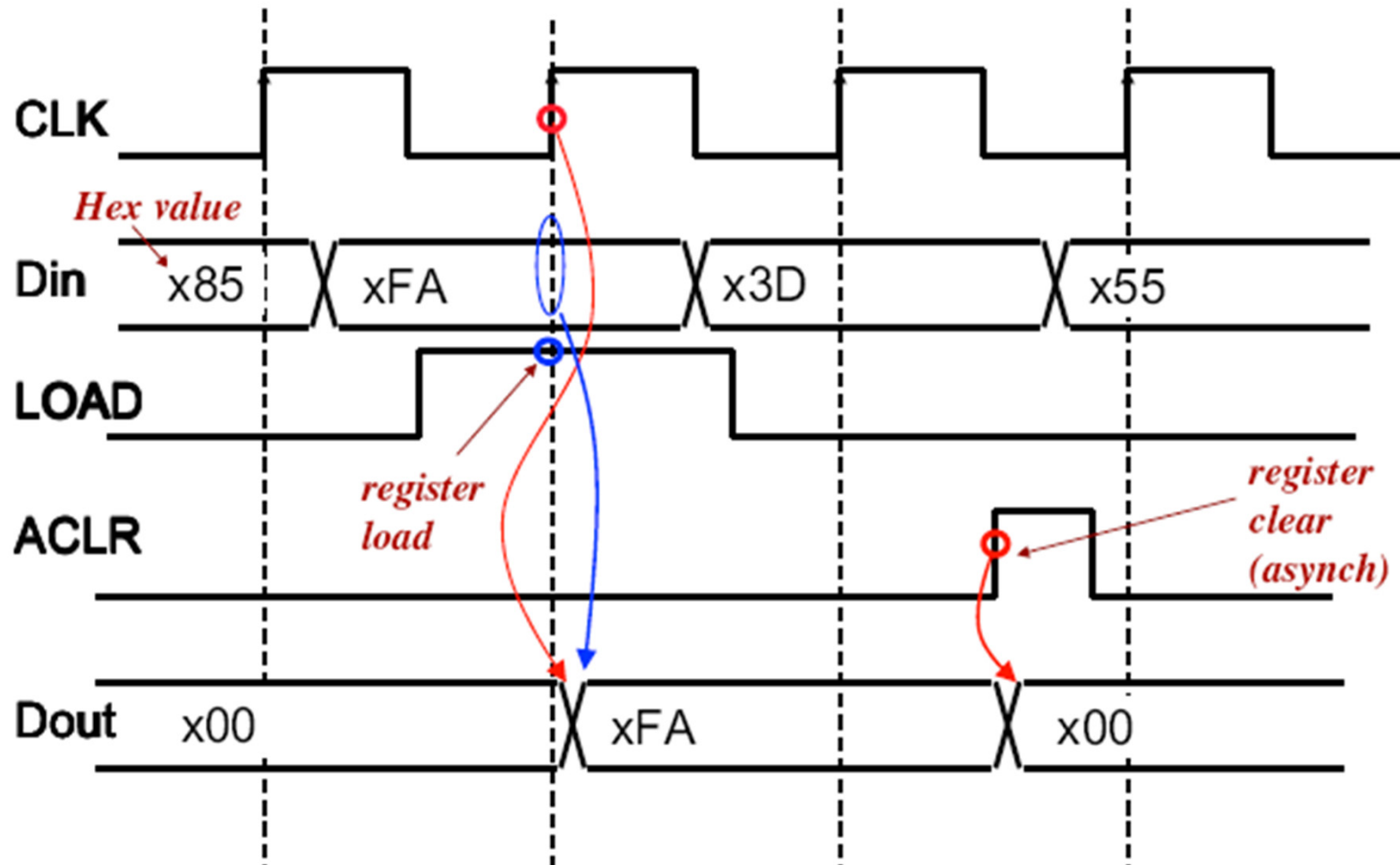


8-bit Register w/LOAD & Asynchronous CLR



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Timing waveform



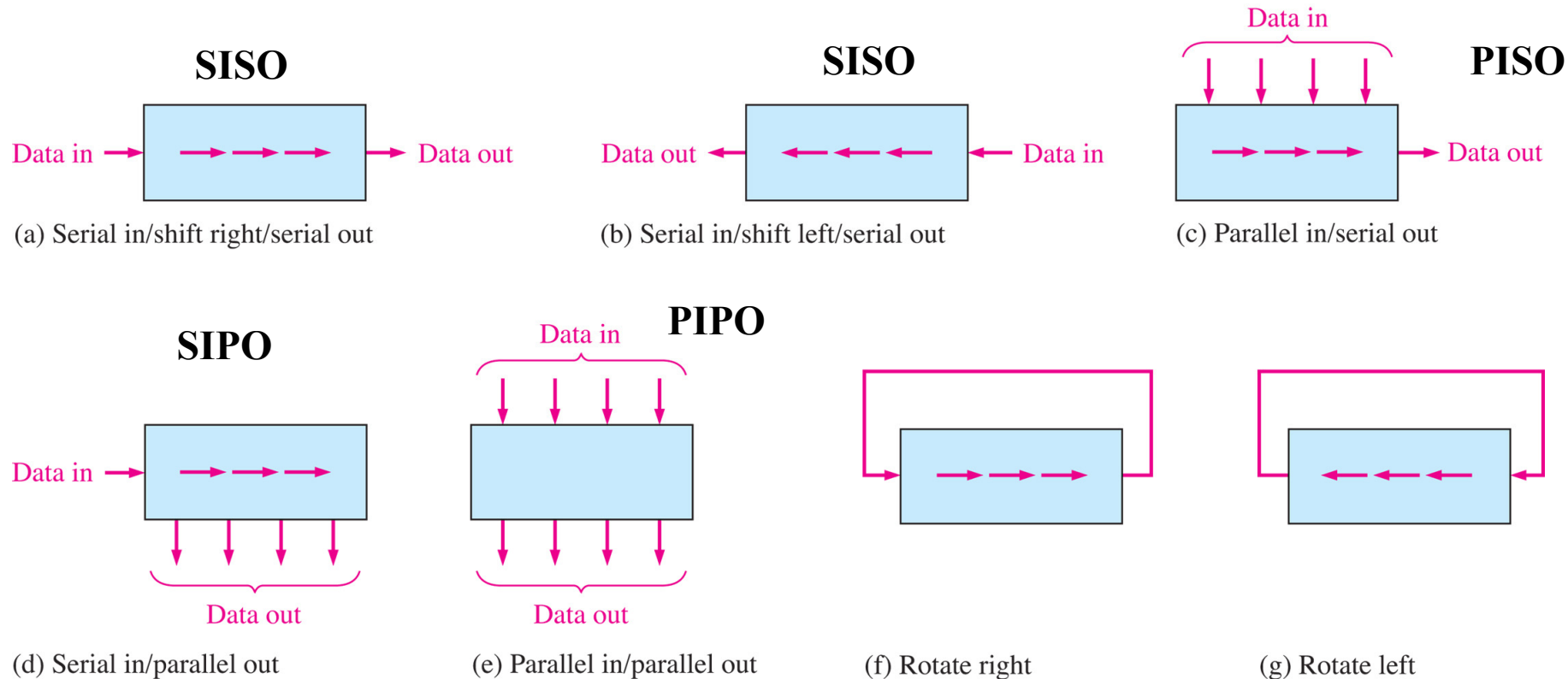
- **Shift registers are a type of sequential logic circuit used primarily for the storage of digital data**
- **Unlike other sequential circuits, no specified internal states (exceptions apply)**
- **The shift capability of a register permits the movement of data from stage to stage within the register or into or out of the register upon application of clock pulses.**

Data Movement in Shift Registers

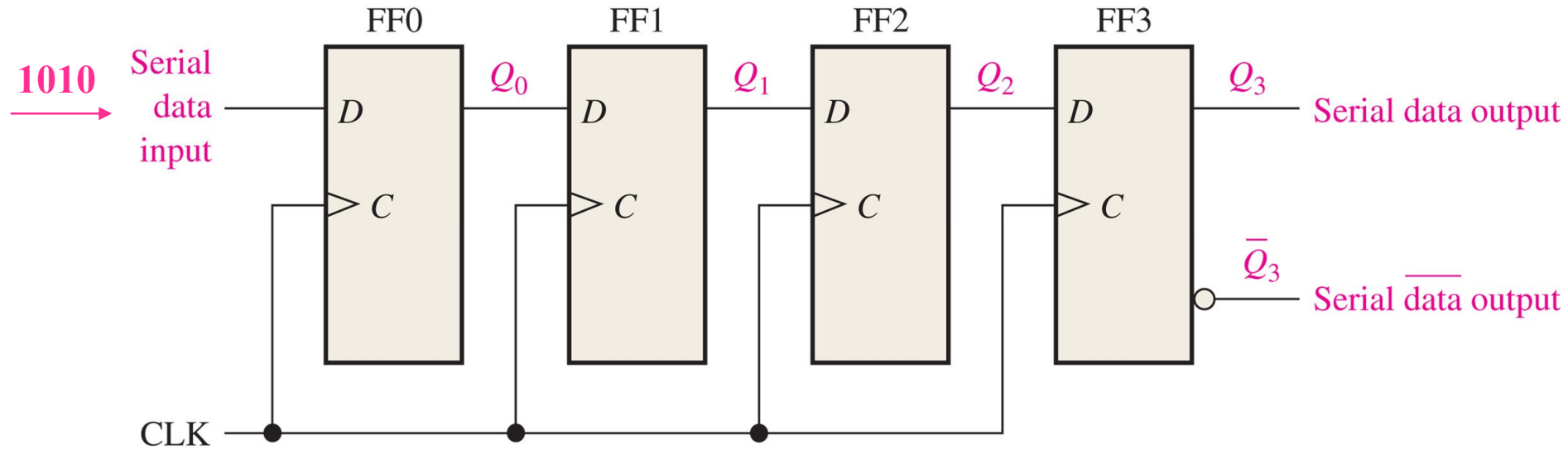


THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY

Four bits are used for illustration.
The bits move in the direction of the arrows.



Serial in/serial out shift register



Loading 4 bits (shifting)

TABLE 8-1

Shifting a 4-bit code into the shift register in Figure 8-3. Data bits are indicated by a beige screen.

CLK	FF0 (Q_0)	FF1 (Q_1)	FF2 (Q_2)	FF3 (Q_3)
Initial	0	0	0	0
1	0	0	0	0
2	1	0	0	0
3	0	1	0	0
4	1	0	1	0

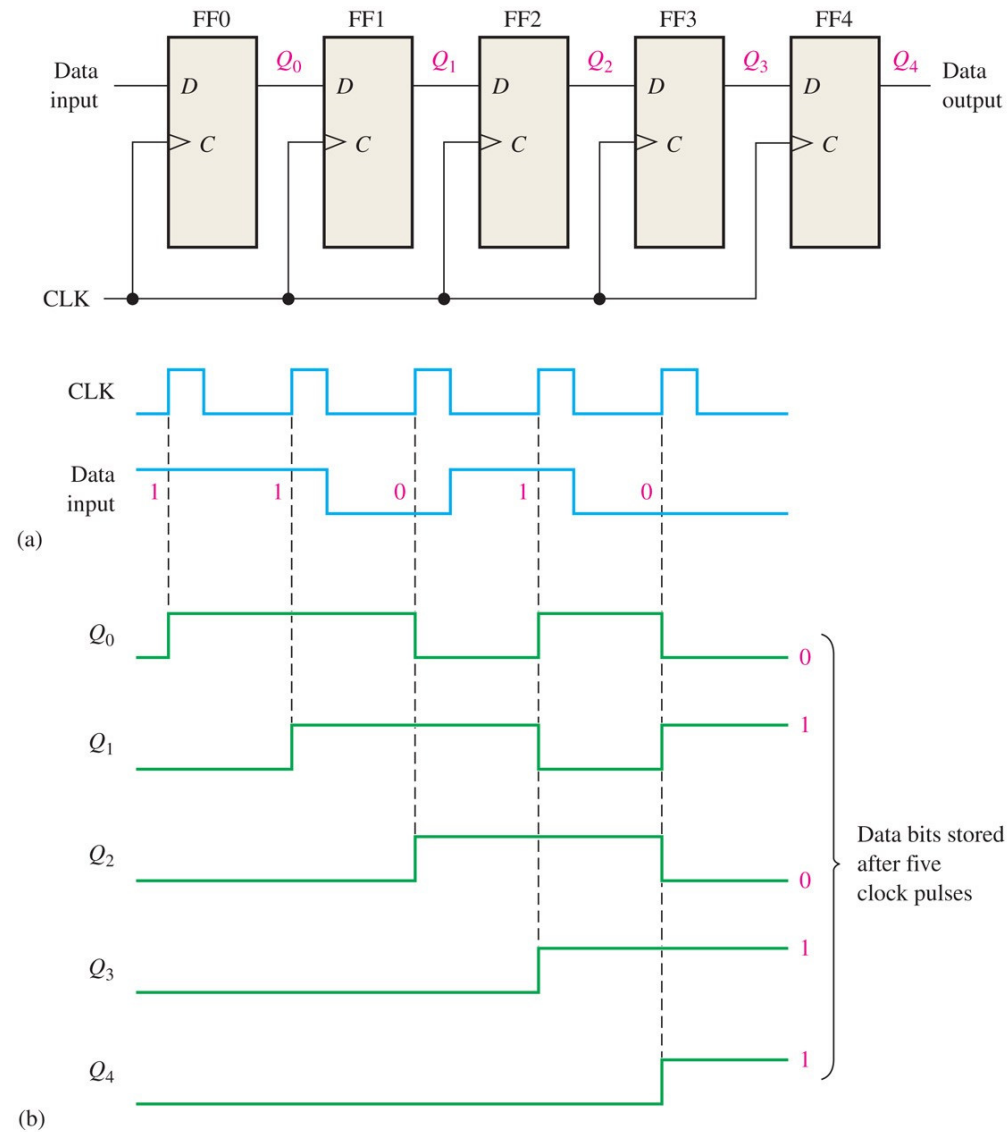
Reading 4 bits (shifting)

TABLE 8-2

Shifting a 4-bit code out of the shift register in Figure 8-3. Data bits are indicated by a beige screen.

CLK	FF0 (Q_0)	FF1 (Q_1)	FF2 (Q_2)	FF3 (Q_3)
Initial	1	0	1	0
5	0	1	0	1
6	0	0	1	0
7	0	0	0	1
8	0	0	0	0

5-bit SISO shift register



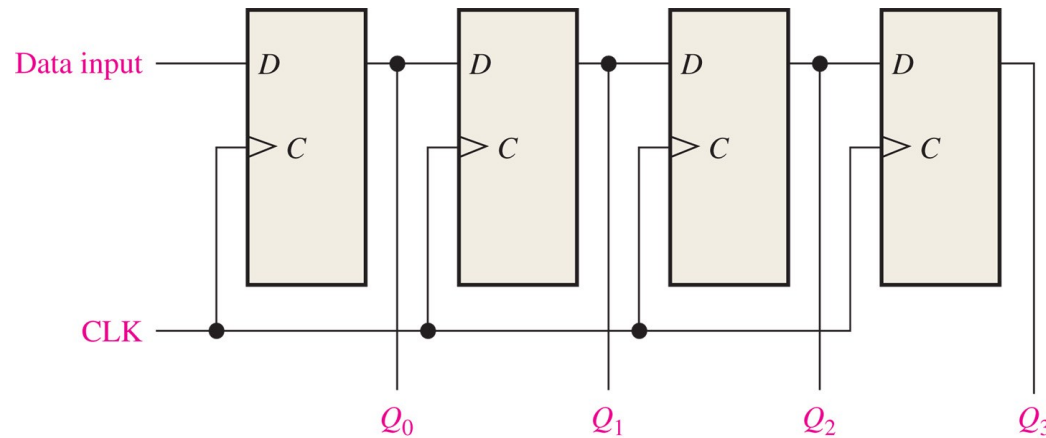
8-bit SISO shift register



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY



Serial in/parallel out shift register

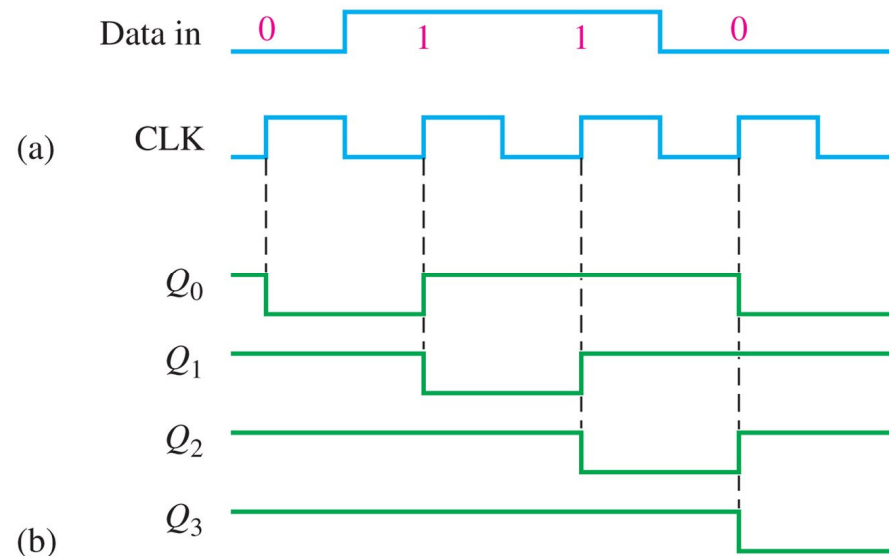


Data bits are entered serially (LSB first)

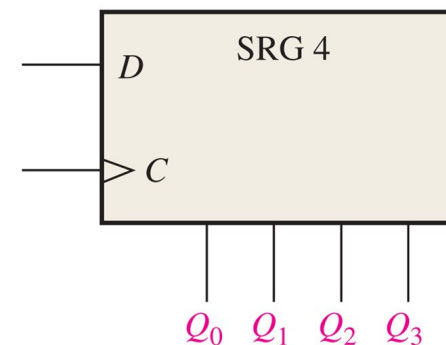
the output of each stage is available.

Once the data are stored, all bits are available simultaneously, rather than on a bit-by-bit basis as with the serial output.

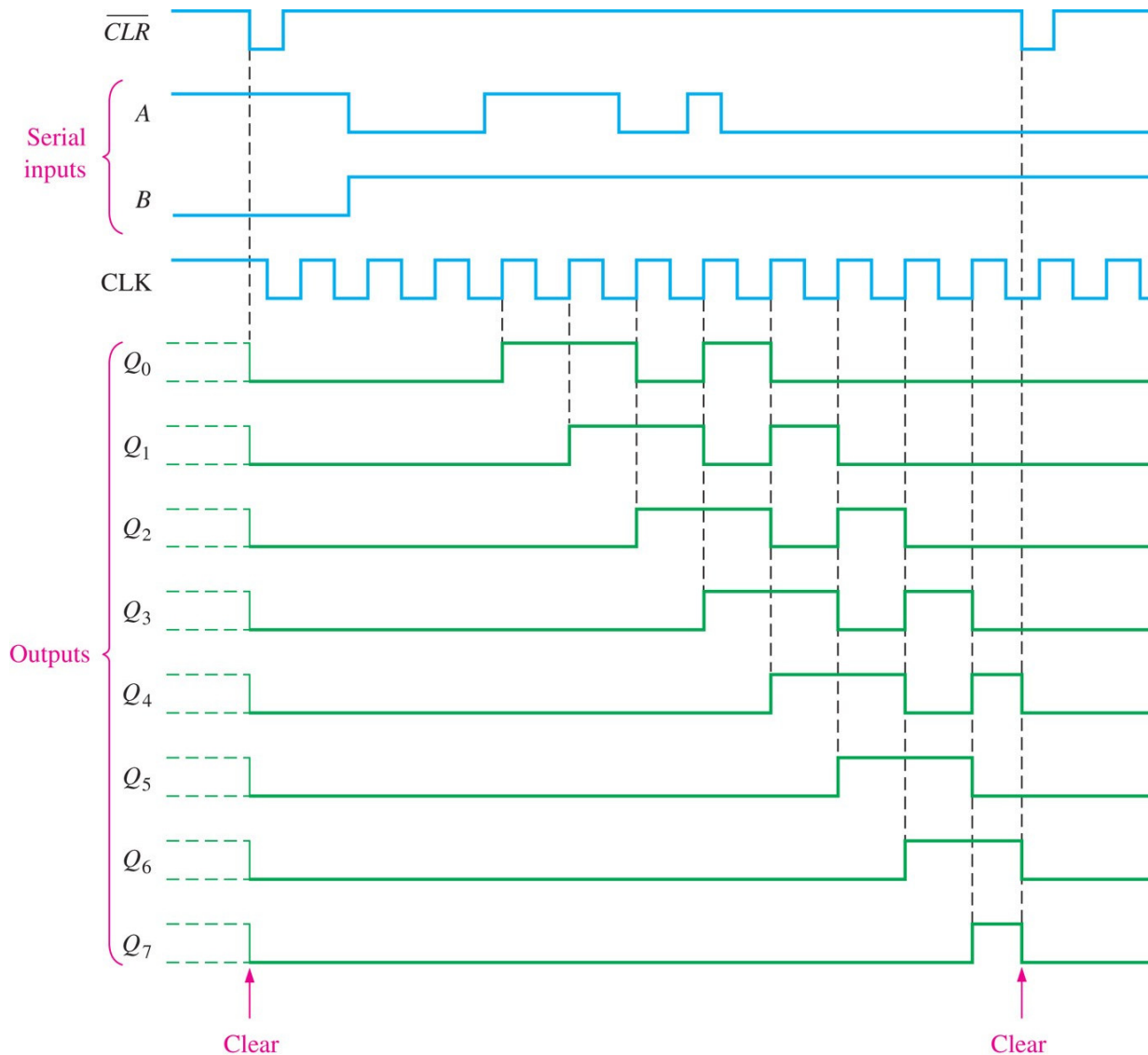
The register contains 0110 after four clock pulses



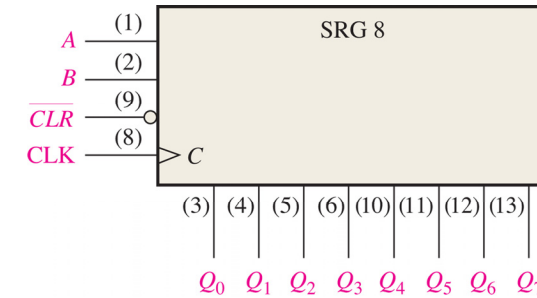
Logic block symbol



Serial in/parallel out shift register



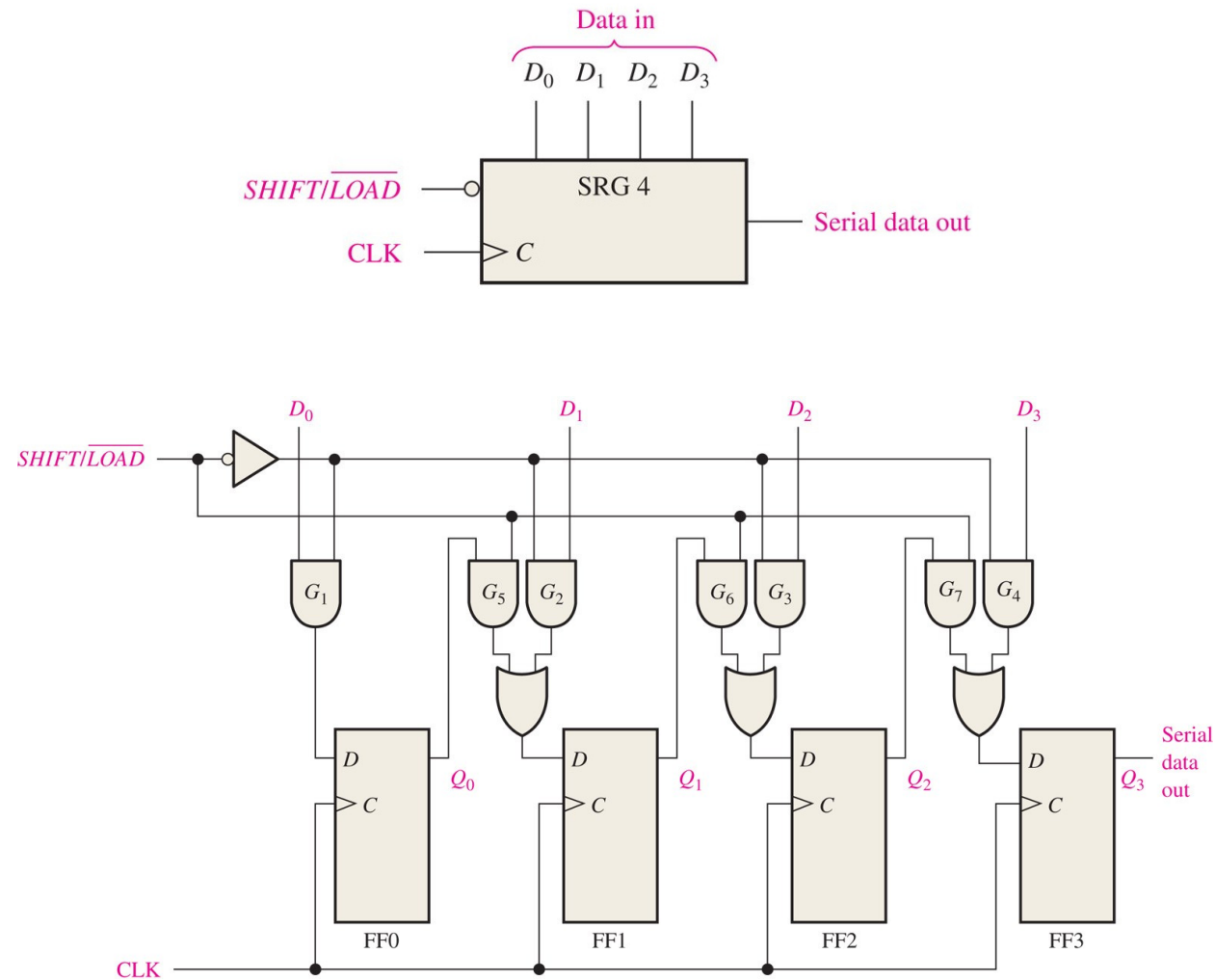
74HC164



This device has two gated serial inputs, A and B, and an asynchronous clear (CLR) input that is active-LOW.

The serial input data on input A are shifted into and through the register after input B goes HIGH

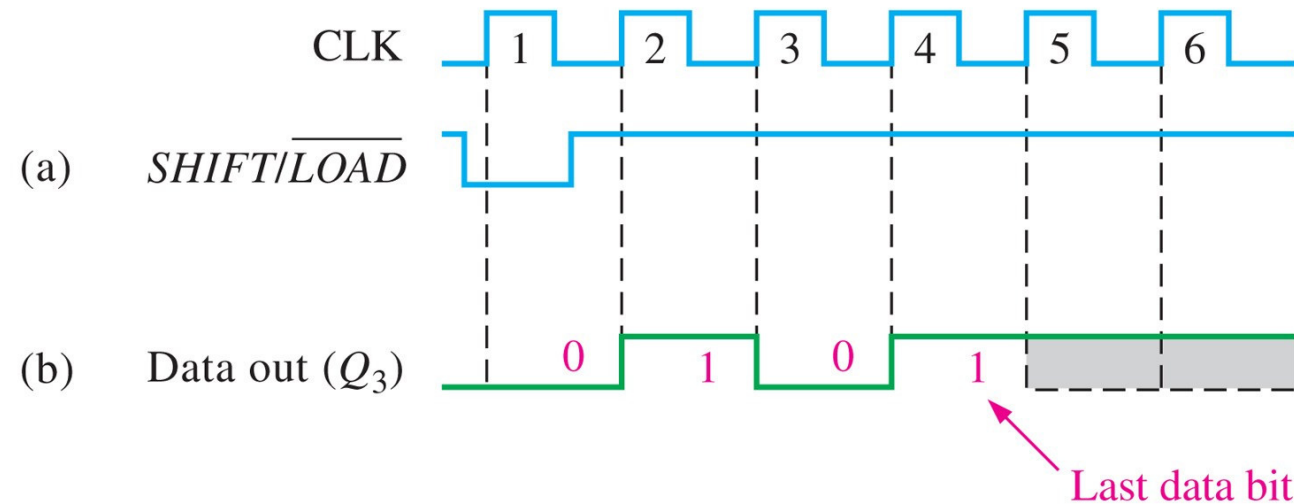
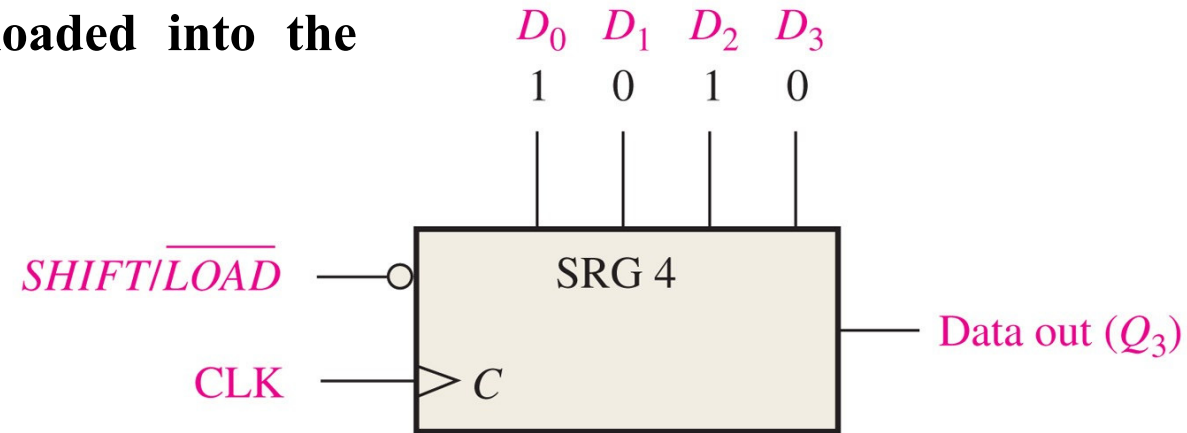
Parallel in/serial out shift register



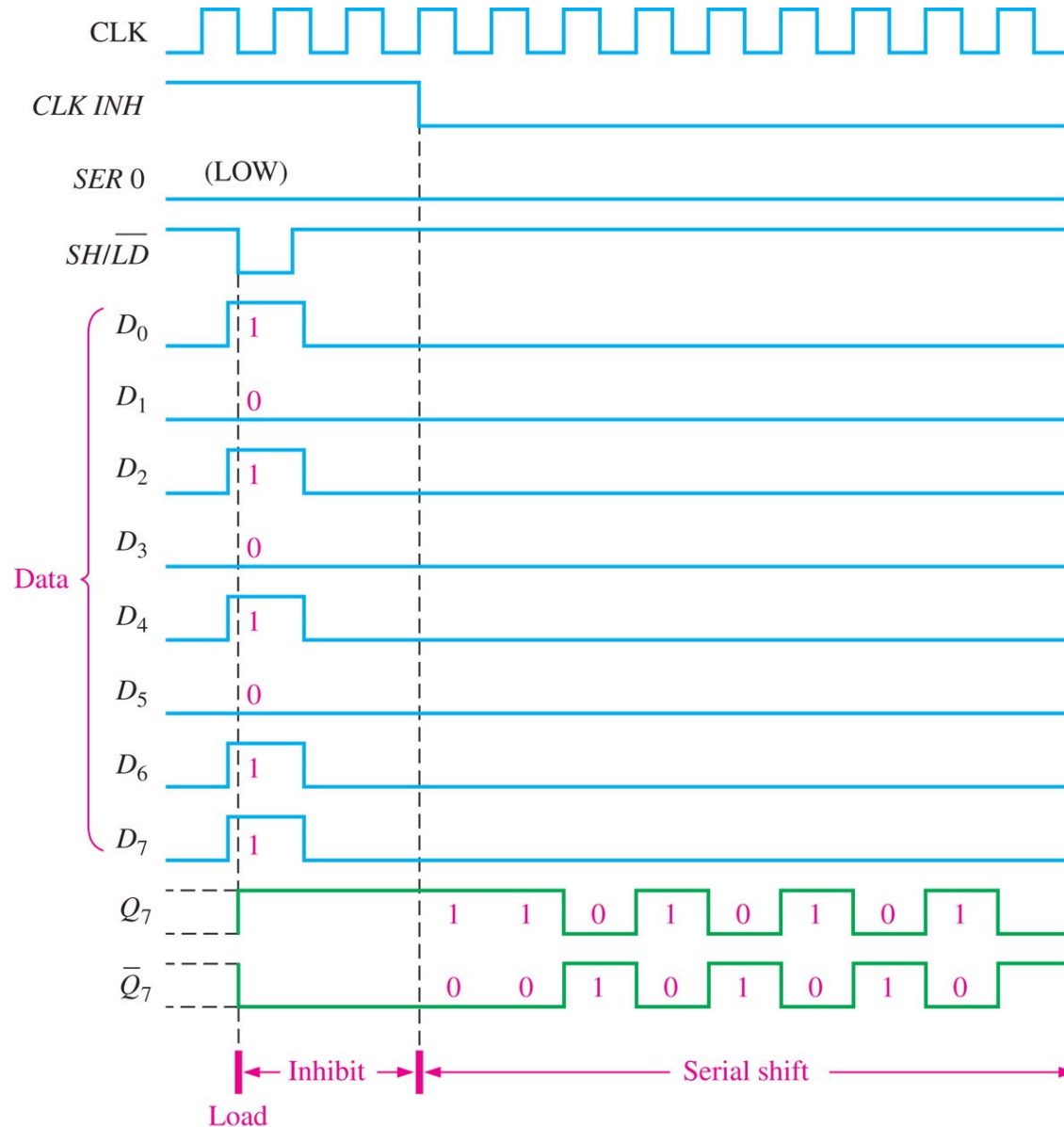
Parallel in/serial out shift register



On clock pulse 1, the parallel data ($D_0D_1D_2D_3 = 1010$) are loaded into the register, making Q_3 a 0.

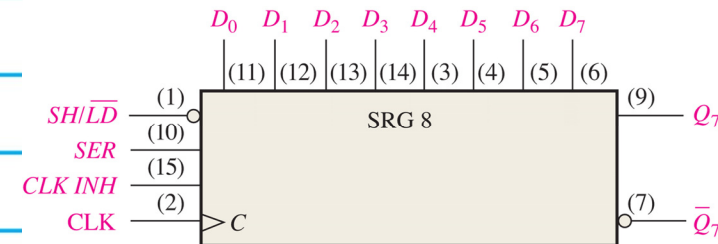


Parallel in/serial out shift register



“Asynchronous” parallel loading

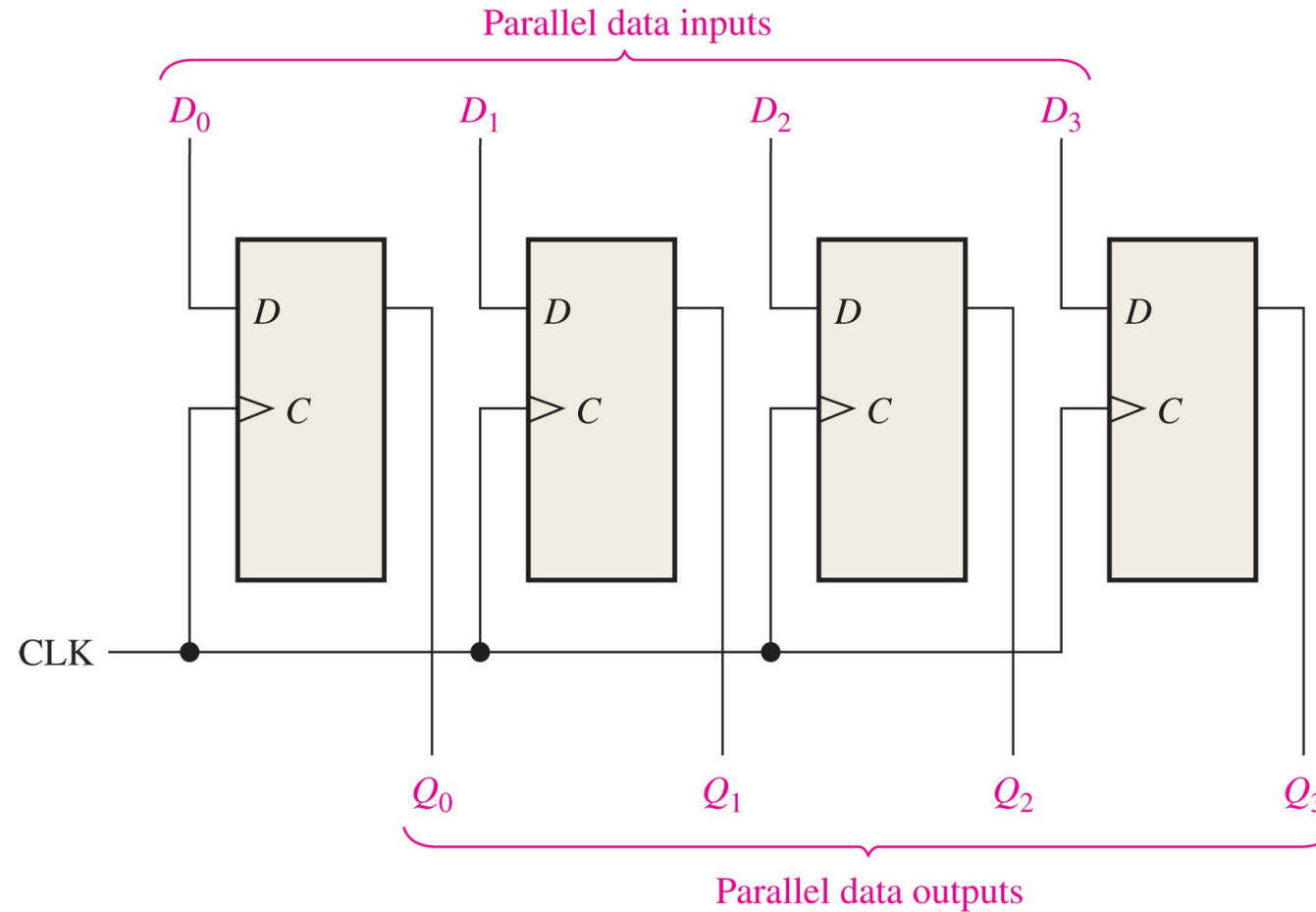
74HC165



Parallel in/parallel out Shift Reg

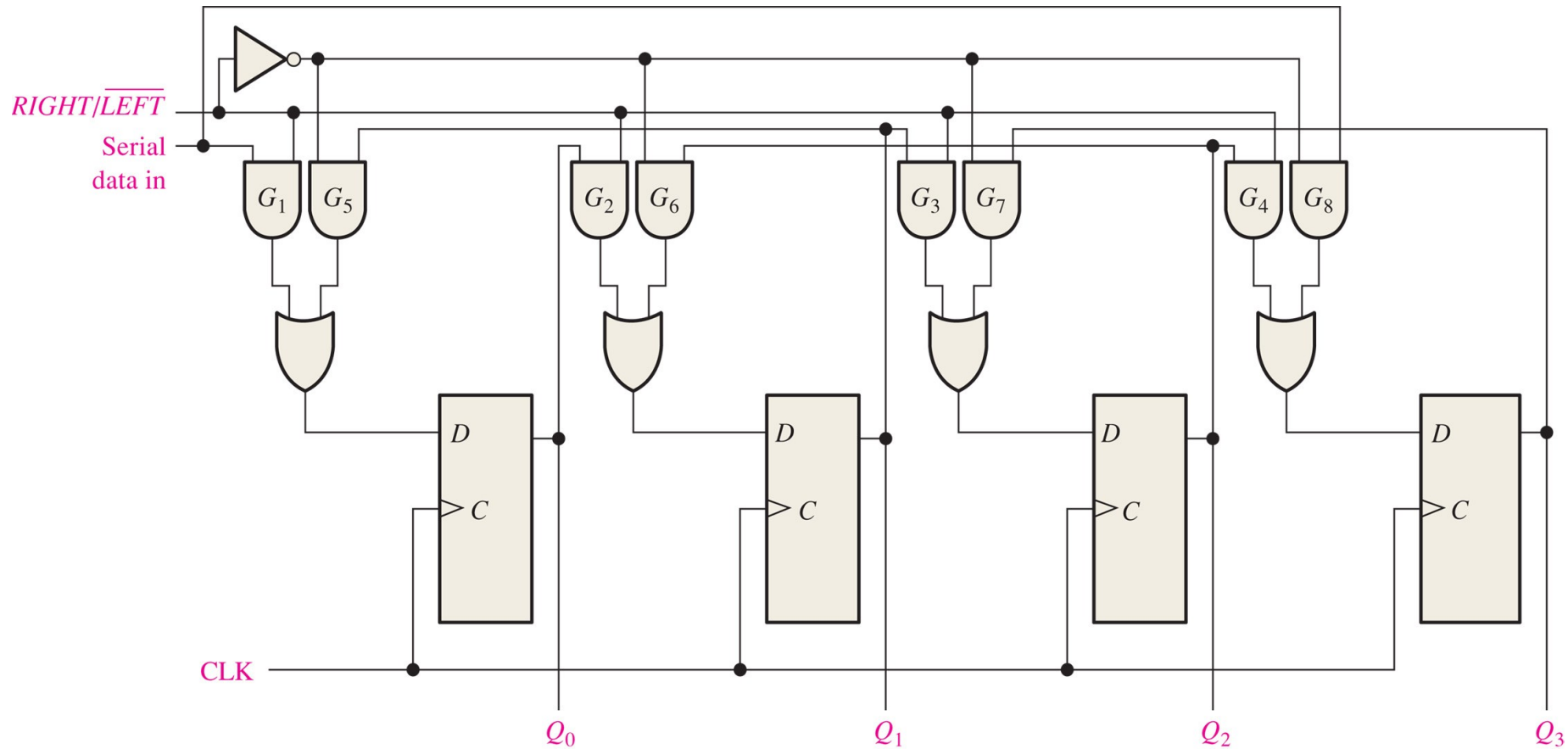


THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY



4-bit bidirectional shift register

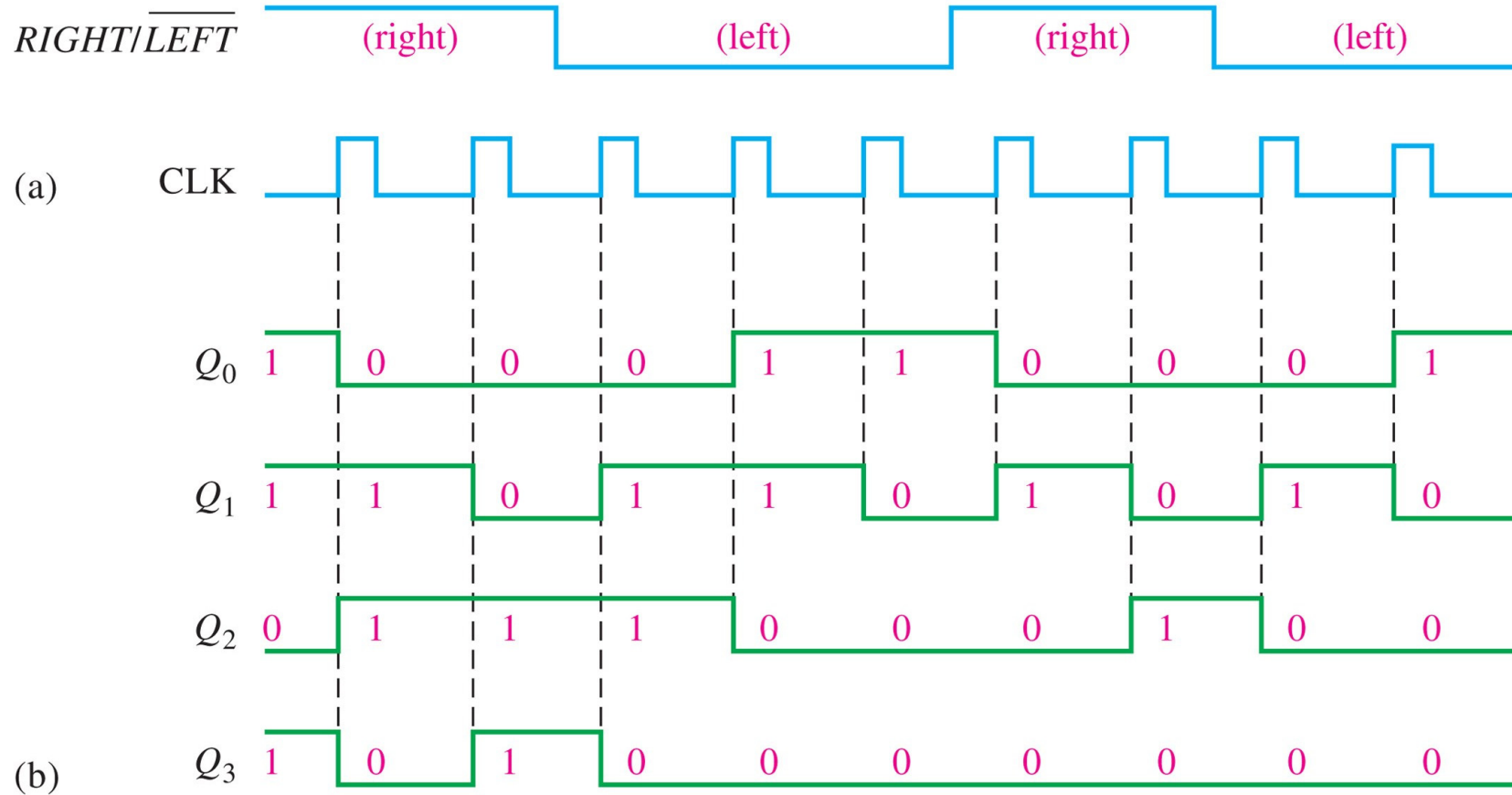
A bidirectional shift register: data can be shifted either left or right.



4-bit bidirectional shift register



Assume that $Q_0 = 1$, $Q_1 = 1$, $Q_2 = 0$, and $Q_3 = 1$ and that the serial data-input line is LOW



4-bit bidirectional universal SR



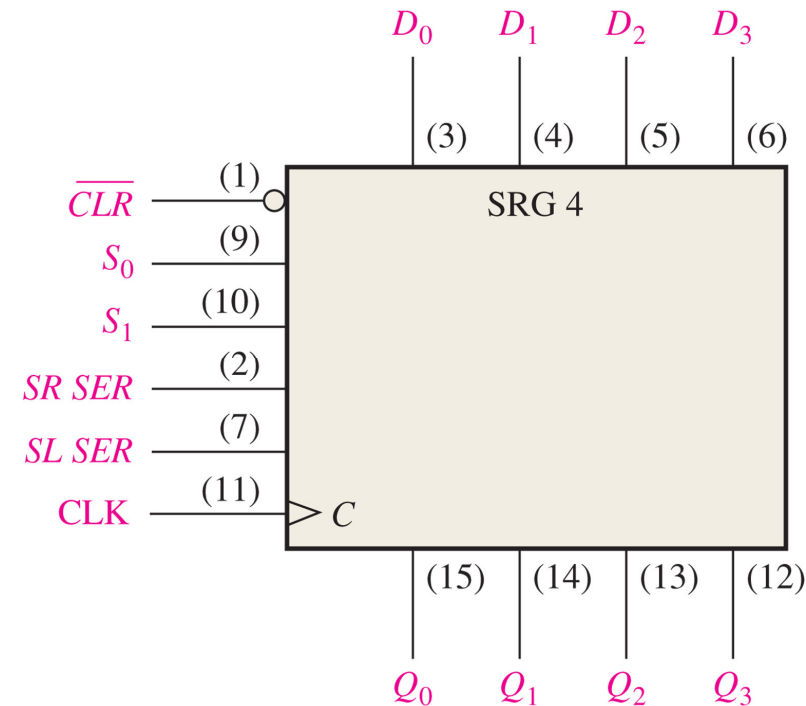
both serial and parallel input and output capability
Support shift in both directions

S_1	S_0	Mode
0	0	Keep Data
0	1	Shift Right
1	0	Shift Left
1	1	Parallel Load

SR SER: serial data input when shift right
it goes into Q_0

SL SER: serial data input when shift left
it goes into Q_3

The 74HC194



Universal Shift Register (Cont' d)



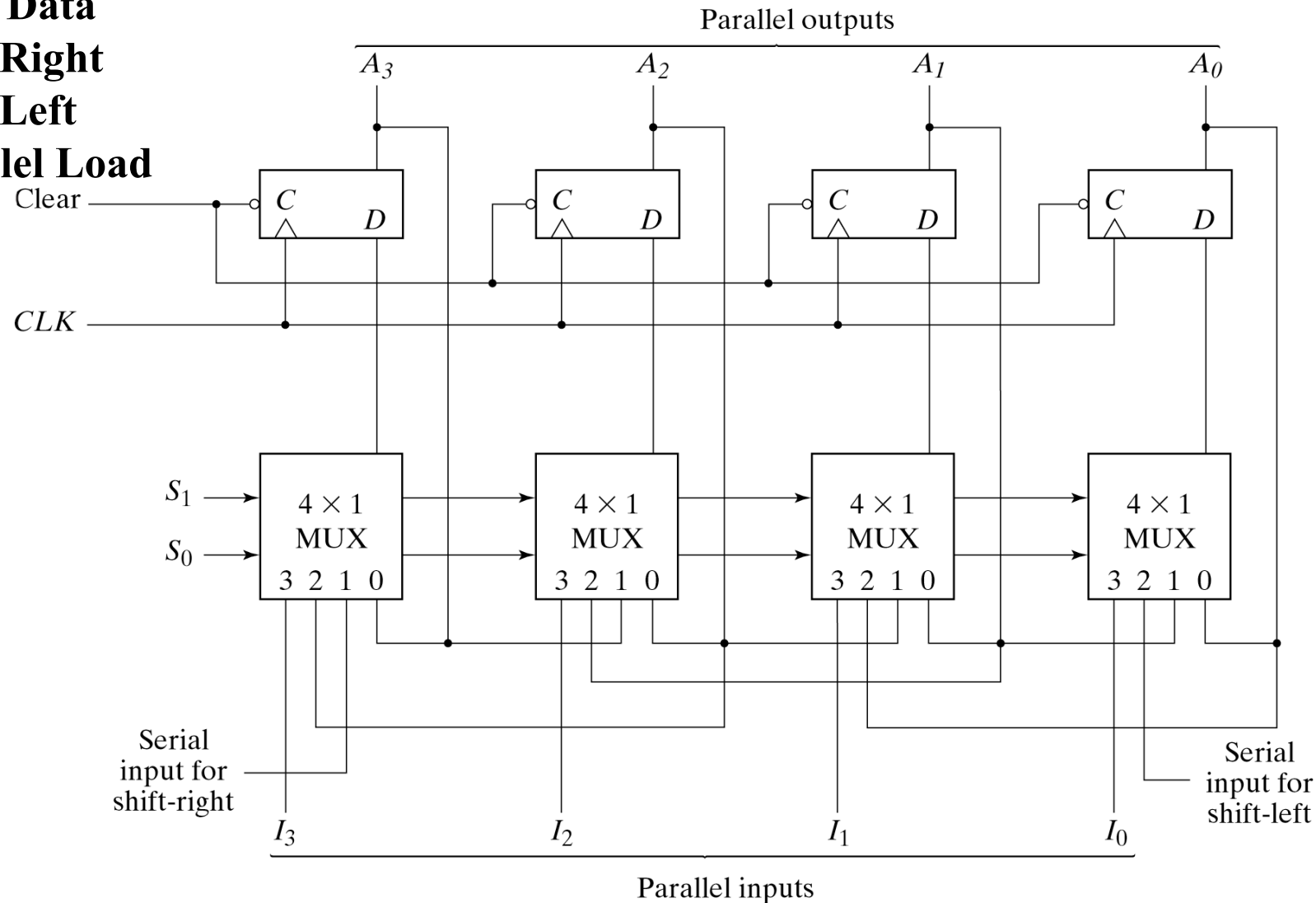
$S_1 S_0$

0 0 → Keep Data

0 1 → Shift Right

1 0 → Shift Left

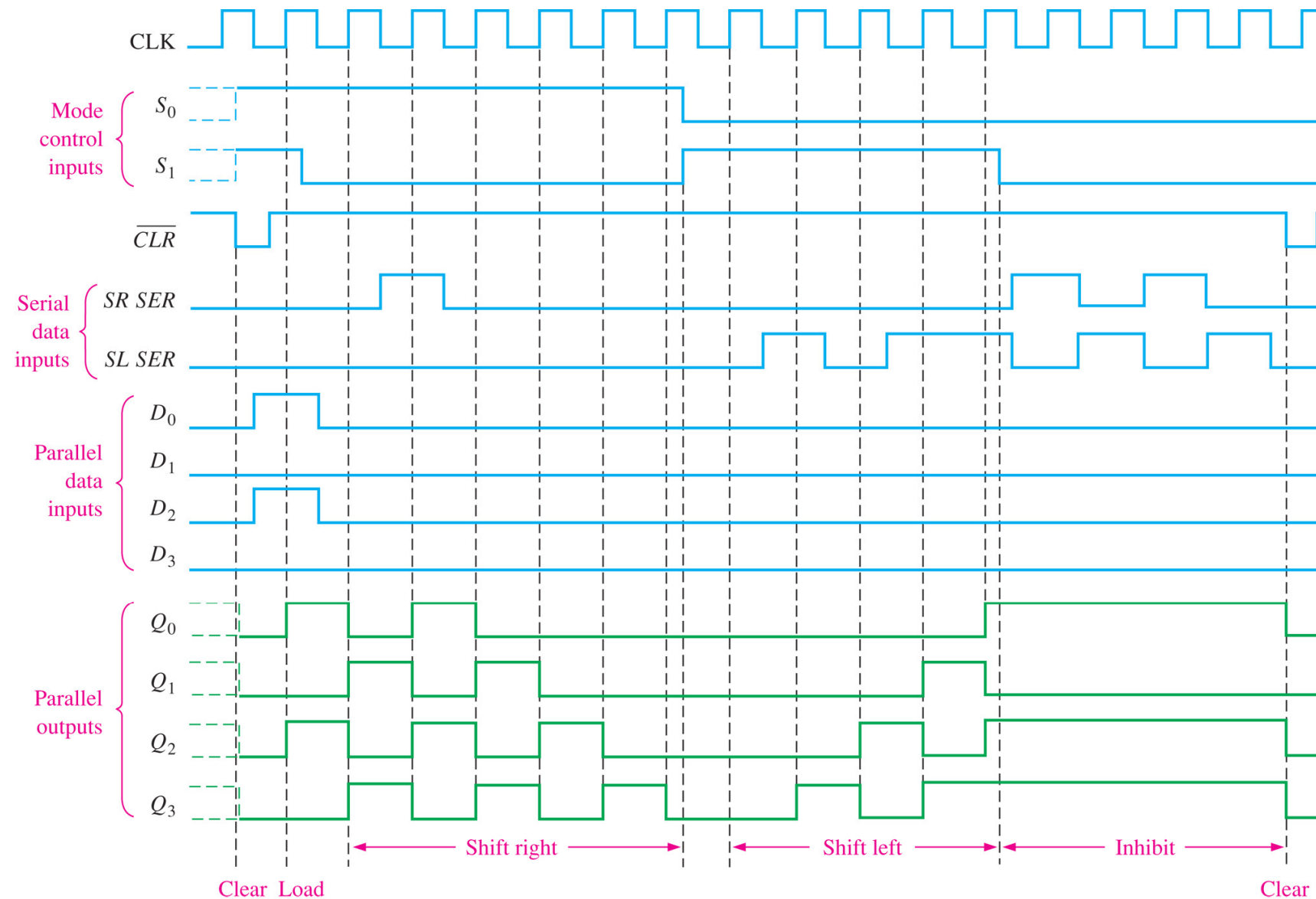
1 1 → Parallel Load



The 74HC194 4-bit bidirectional universal SR



THE HONG KONG
UNIVERSITY OF SCIENCE
AND TECHNOLOGY



- **Serial output connected back to the serial input**
- **Exhibit a specified sequence of states.**
- **Two most common types**
 - 1. the Johnson counter**
 - **the complement output of the last flip-flop is connected back to the D input of the first flip-flop (a.k.a. switch-tail ring counter)**
 - **produce a modulus of $2n$, where n is the number of FFs in the counter (4-bit counter has 8 states)**
 - 2. the ring counter**
 - **one flip-flop for each state in its sequence**
 - **decoding gates are not required**

10-bit Ring Counter

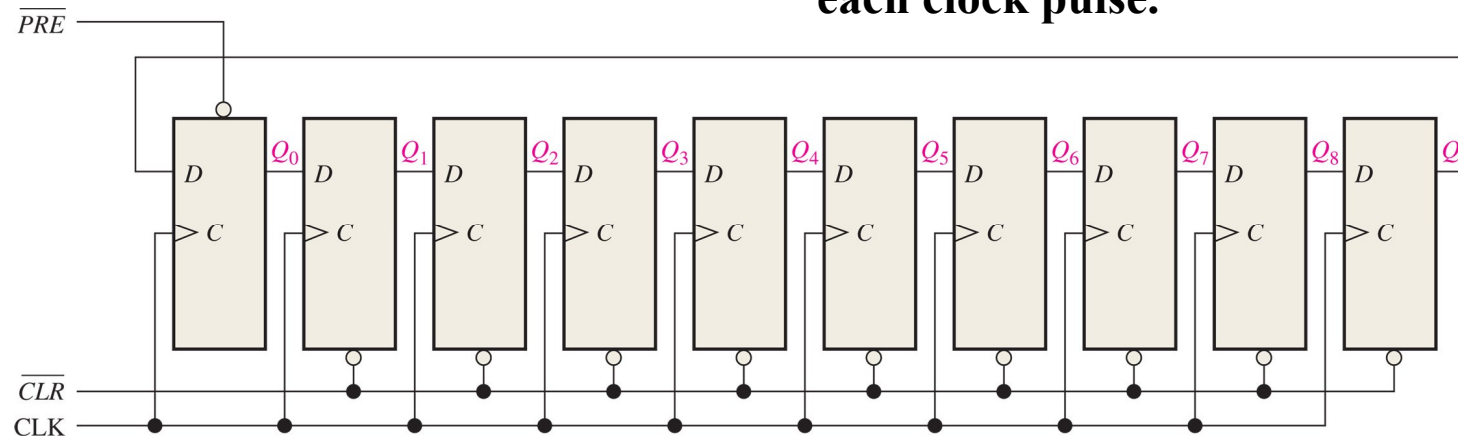
- A ring counter utilizes one flip-flop for each state in its sequence.
- It has the advantage that decoding gates are not required.
- Initially, a 1 is preset into the first flip-flop, and the rest of the flip-flops are cleared.

TABLE 8-5

Ten-bit ring counter sequence.

Clock Pulse	Q_0	Q_1	Q_2	Q_3	Q_4	Q_5	Q_6	Q_7	Q_8	Q_9
0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0
4	0	0	0	0	1	0	0	0	0	0
5	0	0	0	0	0	1	0	0	0	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	0	0	0	0	0	1	0	0
8	0	0	0	0	0	0	0	0	1	0
9	0	0	0	0	0	0	0	0	0	1

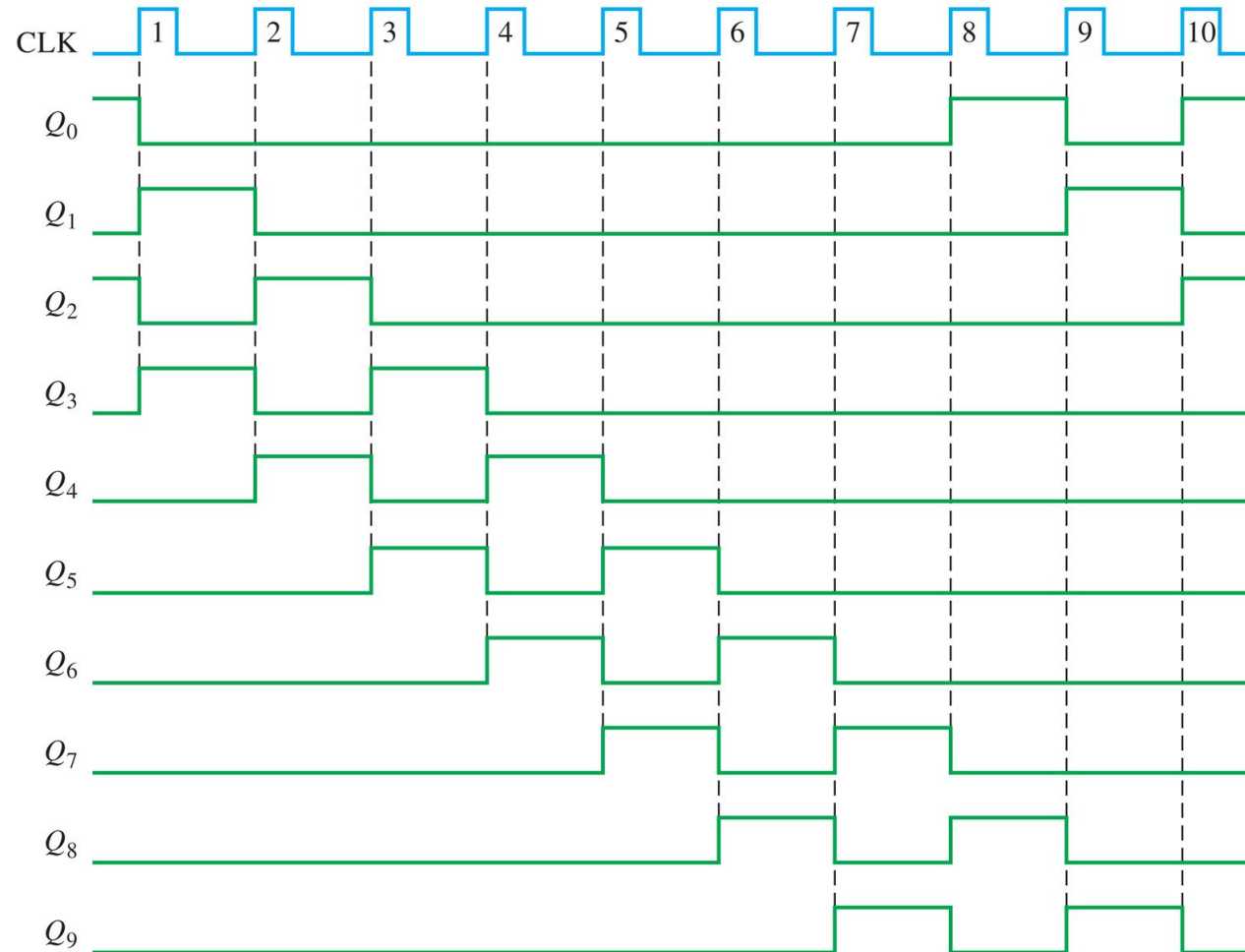
- The ten outputs of the counter indicate directly the decimal count of the clock pulse.
- For instance, a 1 on Q_0 represents a zero, a 1 on Q_1 represents a one, a 1 on Q_2 represents a two, a 1 on Q_3 represents a three, and so on.
- the 1 is always retained in the counter and simply shifted “around the ring,” advancing one stage for each clock pulse.



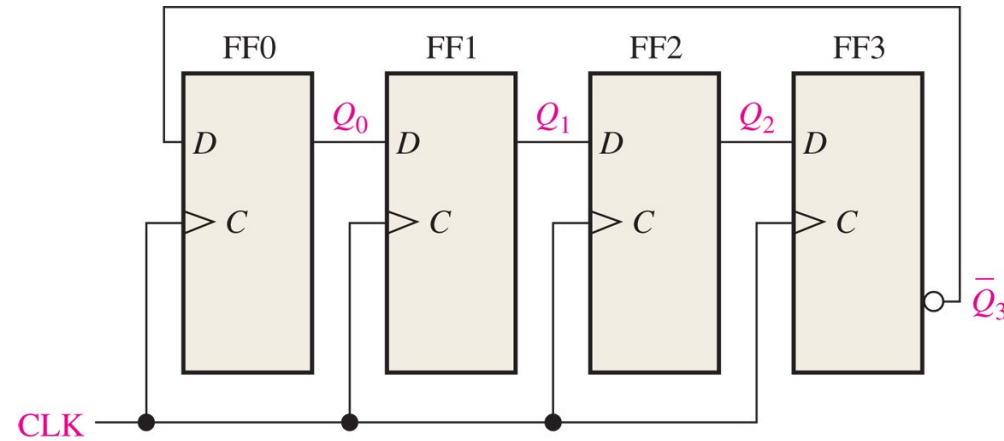
10-bit Ring Counter



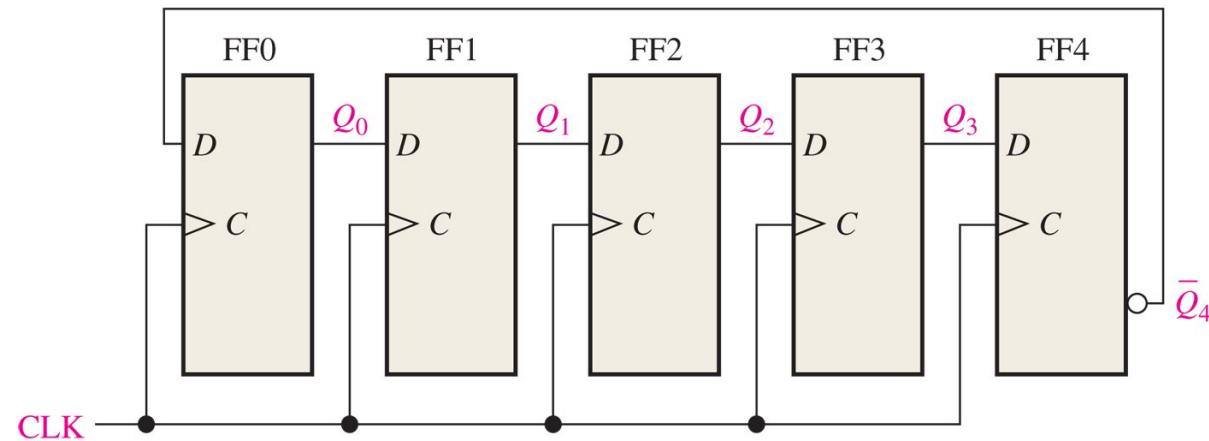
If a 10-bit ring counter has the initial state 1010000000



Johnson Counter



(a) Four-bit Johnson counter



(b) Five-bit Johnson counter

4-bit Johnson Sequence



TABLE 8-3

Four-bit Johnson sequence.

Clock Pulse	Q_0	Q_1	Q_2	Q_3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1

5-bit Johnson Sequence

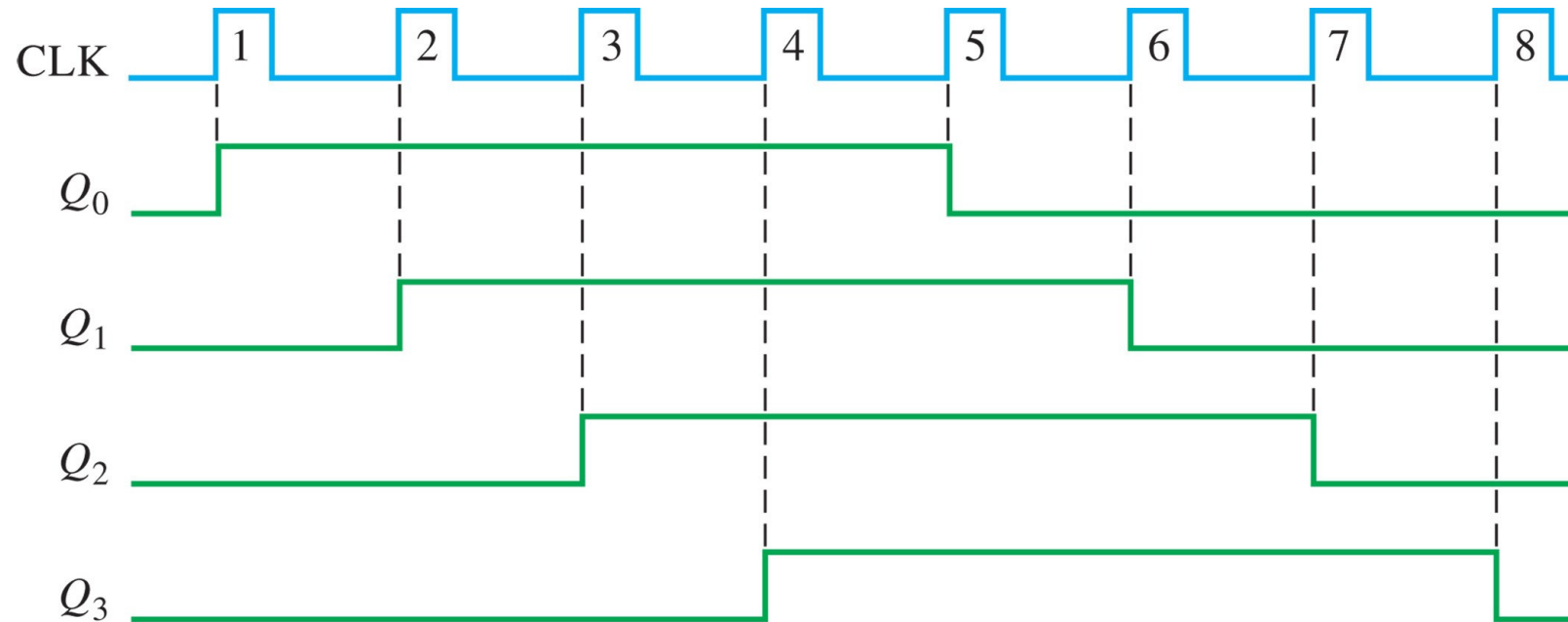


TABLE 8–4

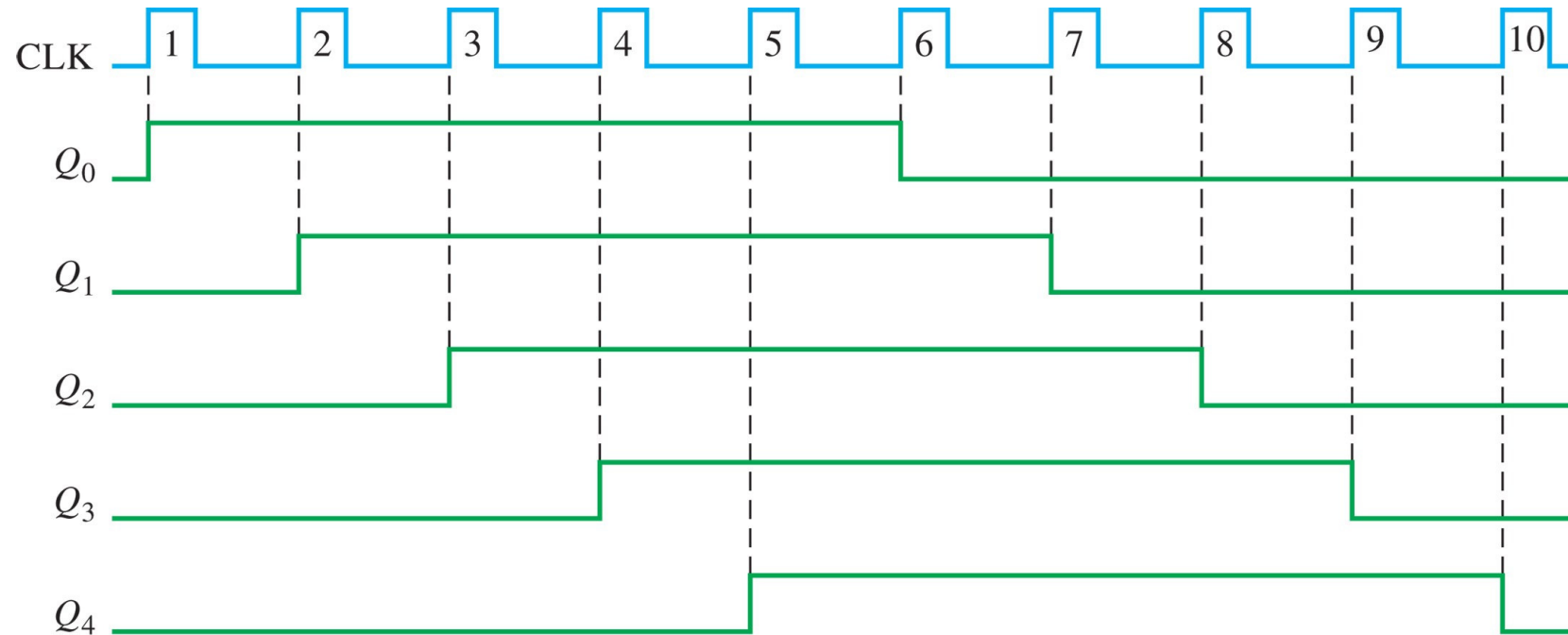
Five-bit Johnson sequence.

Clock Pulse	Q_0	Q_1	Q_2	Q_3	Q_4
0	0	0	0	0	0
1	1	0	0	0	0
2	1	1	0	0	0
3	1	1	1	0	0
4	1	1	1	1	0
5	1	1	1	1	1
6	0	1	1	1	1
7	0	0	1	1	1
8	0	0	0	1	1
9	0	0	0	0	1

4-bit Johnson Counter



5-bit Johnson Counter



4-bit Johnson Counter w/ a decoder

A 4-bit Johnson Counter requires 8 decoding gates to produce 8 timing signals

Pulse	Q_3	Q_2	Q_1	Q_0	Output Y_i
	0	0	0	0	$Y_7 = \overline{Q_3} \cdot \overline{Q_0}$
1	0	0	0	1	$Y_6 = \overline{Q_1} \cdot Q_0$
2	0	0	1	1	$Y_5 = \overline{Q_2} \cdot Q_1$
3	0	1	1	1	$Y_4 = \overline{Q_3} \cdot Q_2$
4	1	1	1	1	$Y_3 = Q_3 \cdot Q_0$
5	1	1	1	0	$Y_2 = Q_1 \cdot \overline{Q_0}$
6	1	1	0	0	$Y_1 = Q_2 \cdot \overline{Q_1}$
7	1	0	0	0	$Y_0 = Q_3 \cdot \overline{Q_2}$

Table 3.5. Count sequence and decoding equations for the four-bit Johnson counter

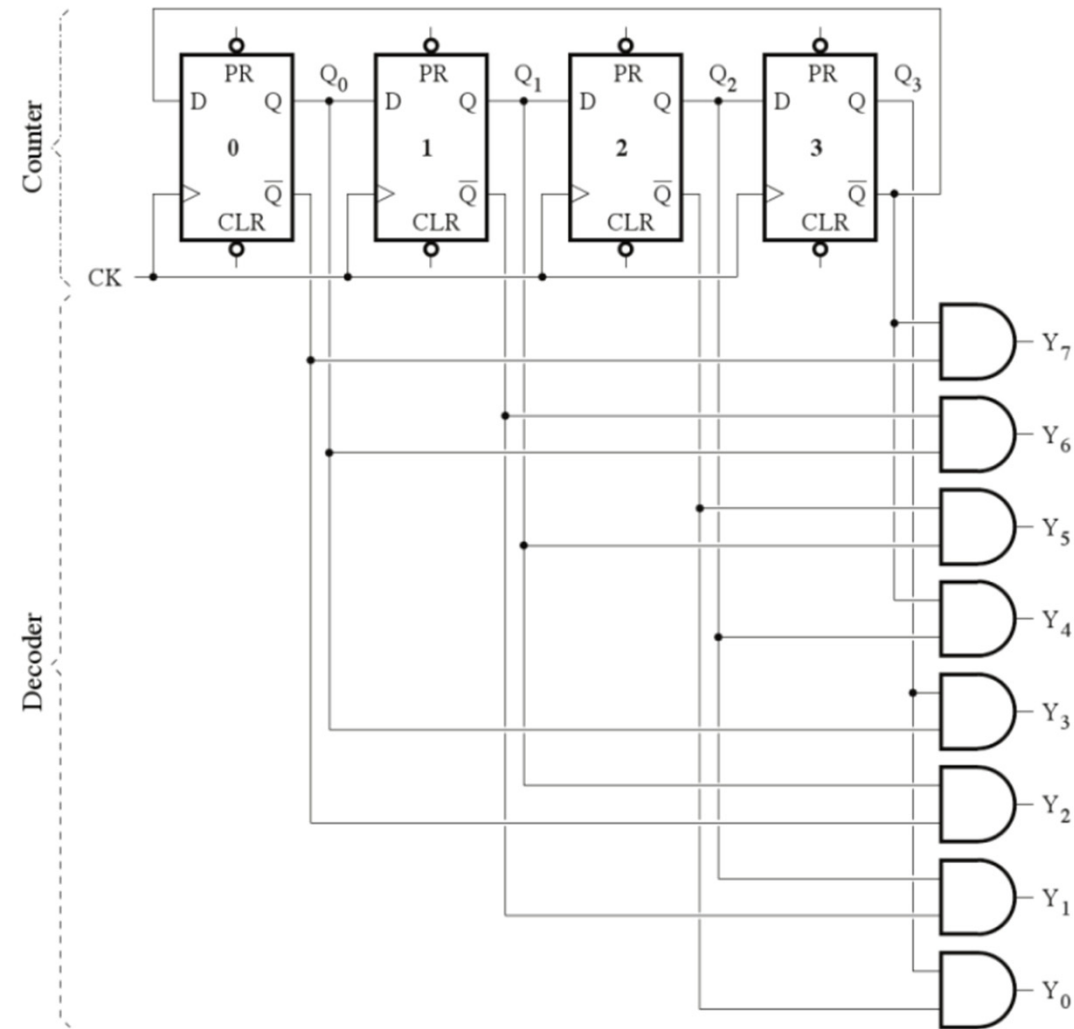


Figure 3.12. Four-bit Johnson counter with a decoder

4-bit Johnson Counter w/ a decoder



**A 4-bit Johnson Counter
requires 8 decoding gates to
produce 8 timing signals**

Pulse	Q_3	Q_2	Q_1	Q_0	Output Y_i
	0	0	0	0	$Y_7 = \overline{Q_3} \cdot \overline{Q_0}$
1	0	0	0	1	$Y_6 = \overline{Q_1} \cdot Q_0$
2	0	0	1	1	$Y_5 = \overline{Q_2} \cdot Q_1$
3	0	1	1	1	$Y_4 = \overline{Q_3} \cdot Q_2$
4	1	1	1	1	$Y_3 = Q_3 \cdot Q_0$
5	1	1	1	0	$Y_2 = Q_1 \cdot \overline{Q_0}$
6	1	1	0	0	$Y_1 = Q_2 \cdot \overline{Q_1}$
7	1	0	0	0	$Y_0 = Q_3 \cdot \overline{Q_2}$

Table 3.5. Count sequence and decoding equations
for the four-bit Johnson counter

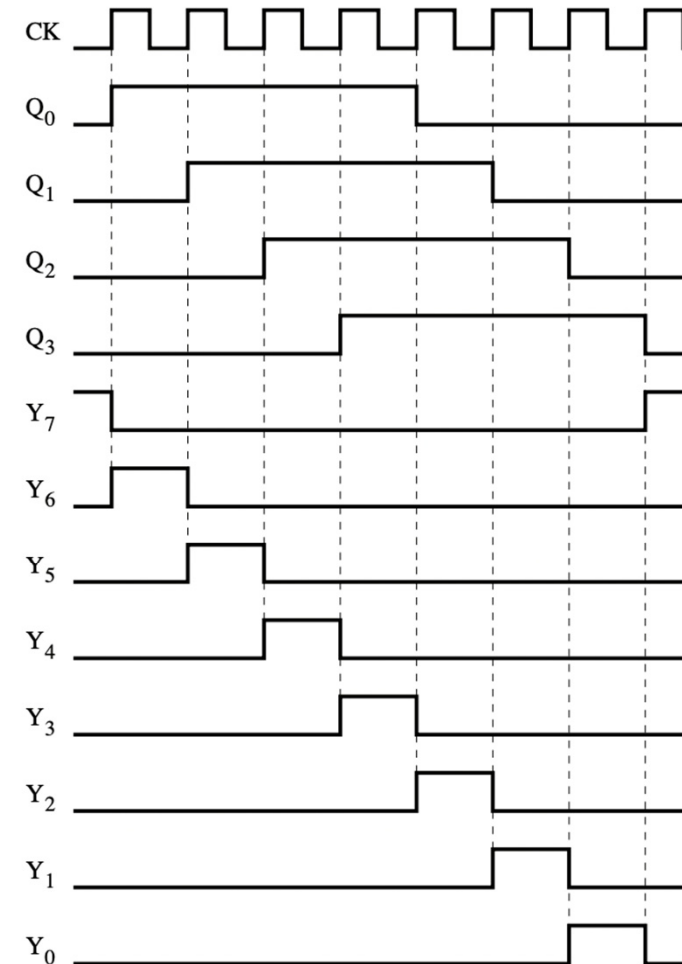
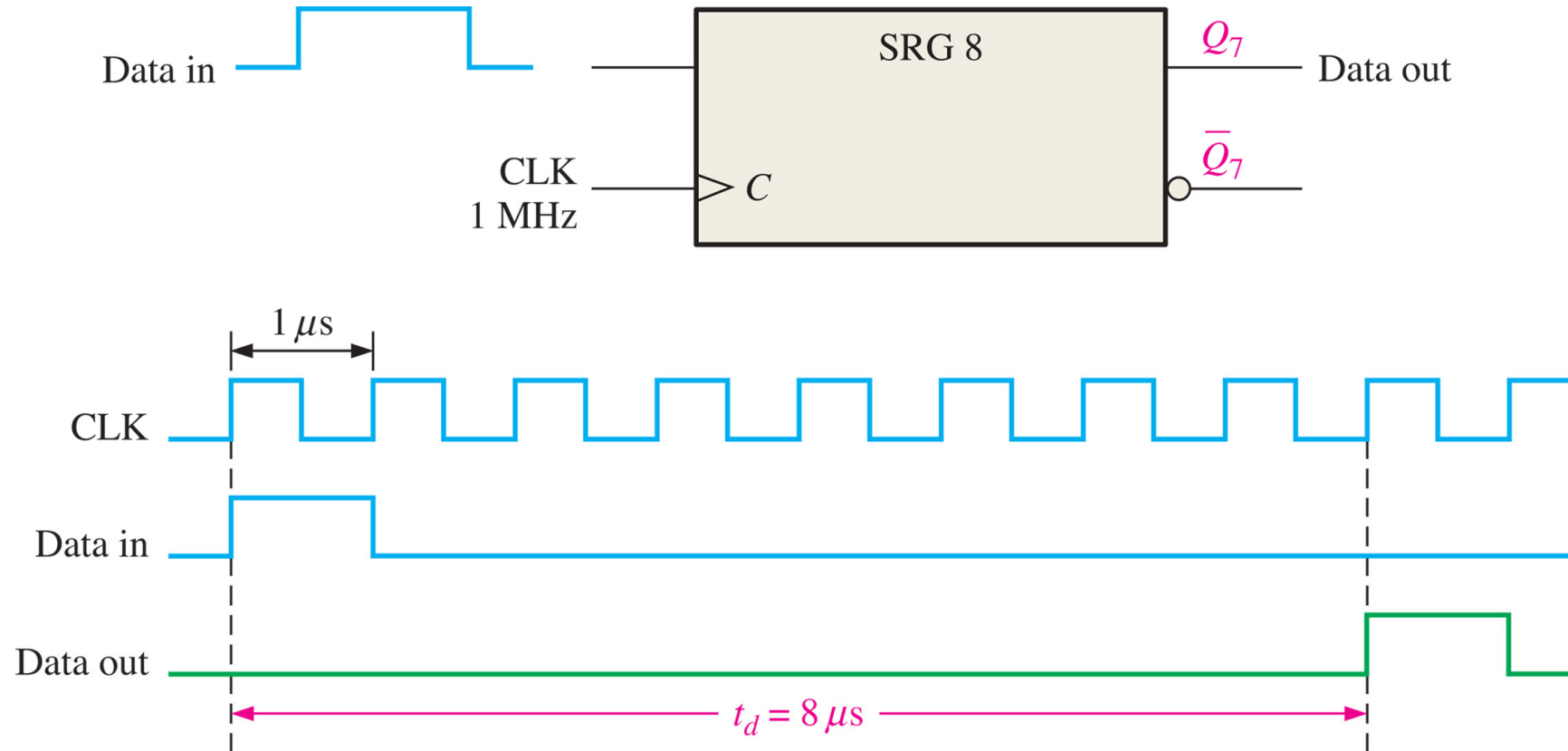


Figure 3.13. Timing diagram of the four-bit
Johnson counter with a decoder

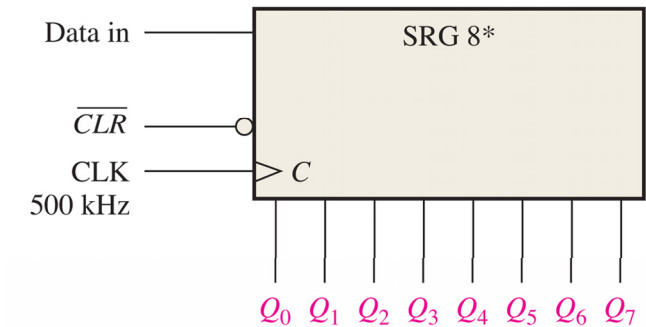
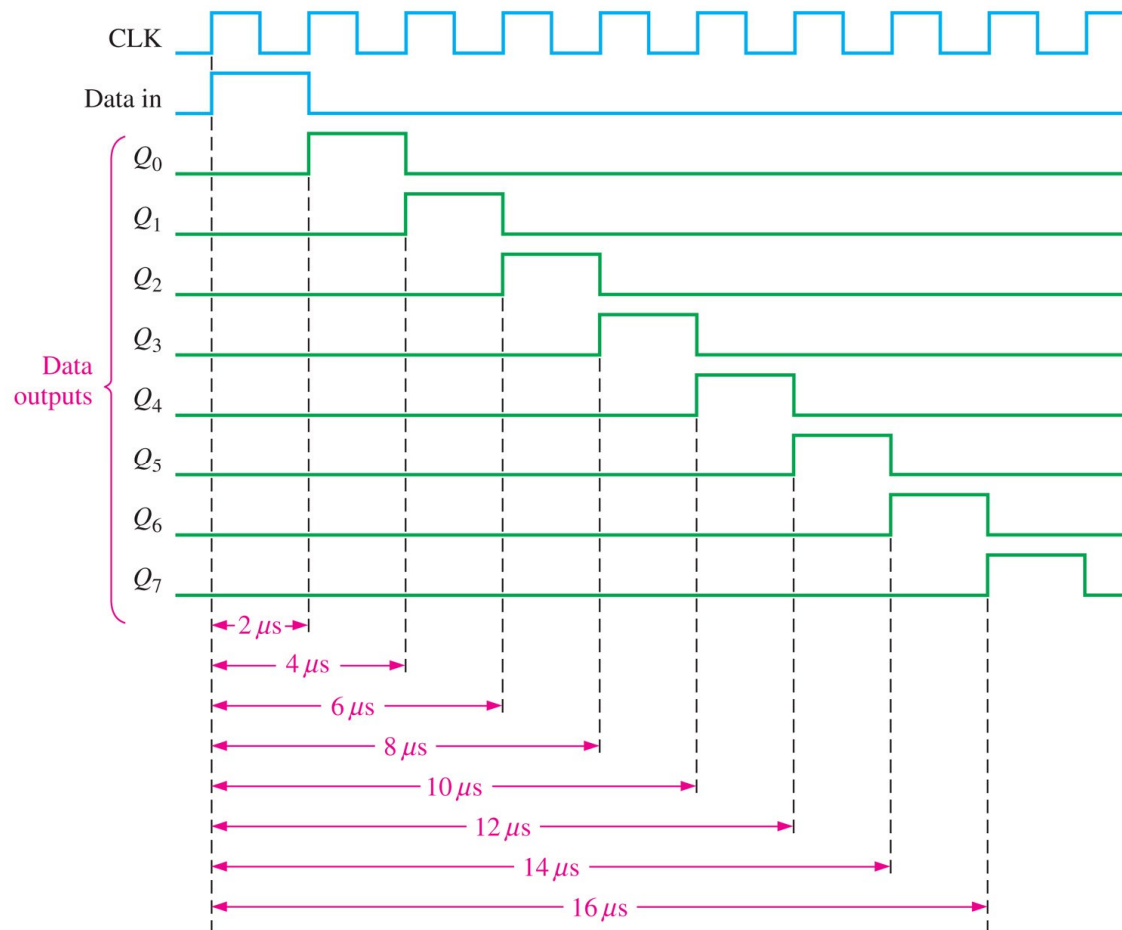
Shift Reg. as a time delay device

A serial in/serial out shift register can be used to provide a time delay from input to output that is a function of both the number of stages (n) in the register and the clock frequency.



Shift Reg. as a time delay device

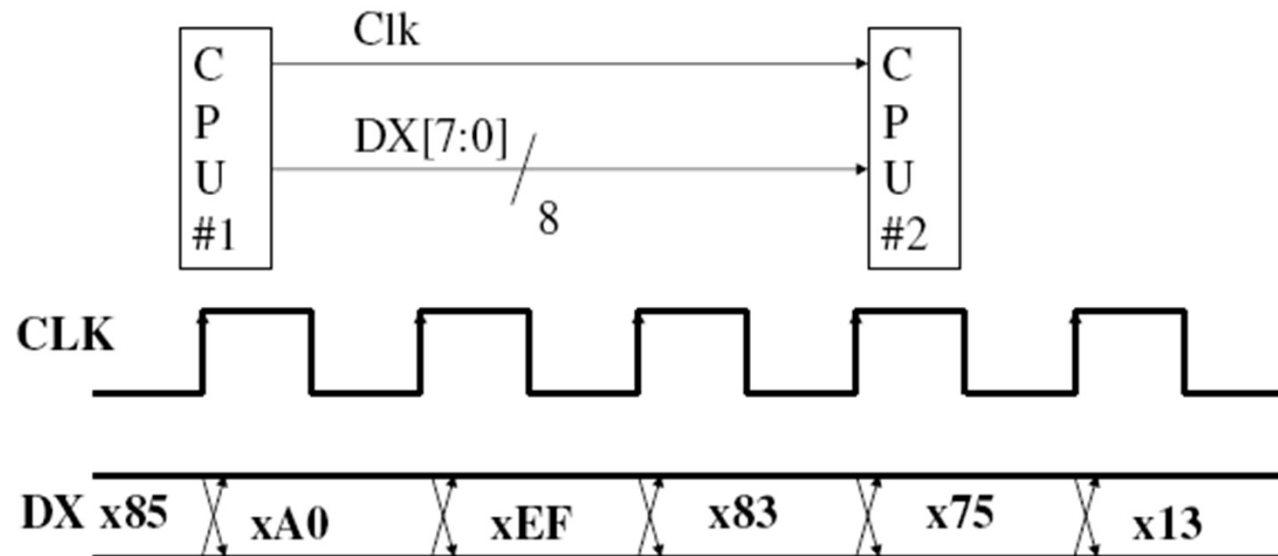
The amount of time delay between the serial input and each output



* Data shifts from Q_0 toward Q_7 .

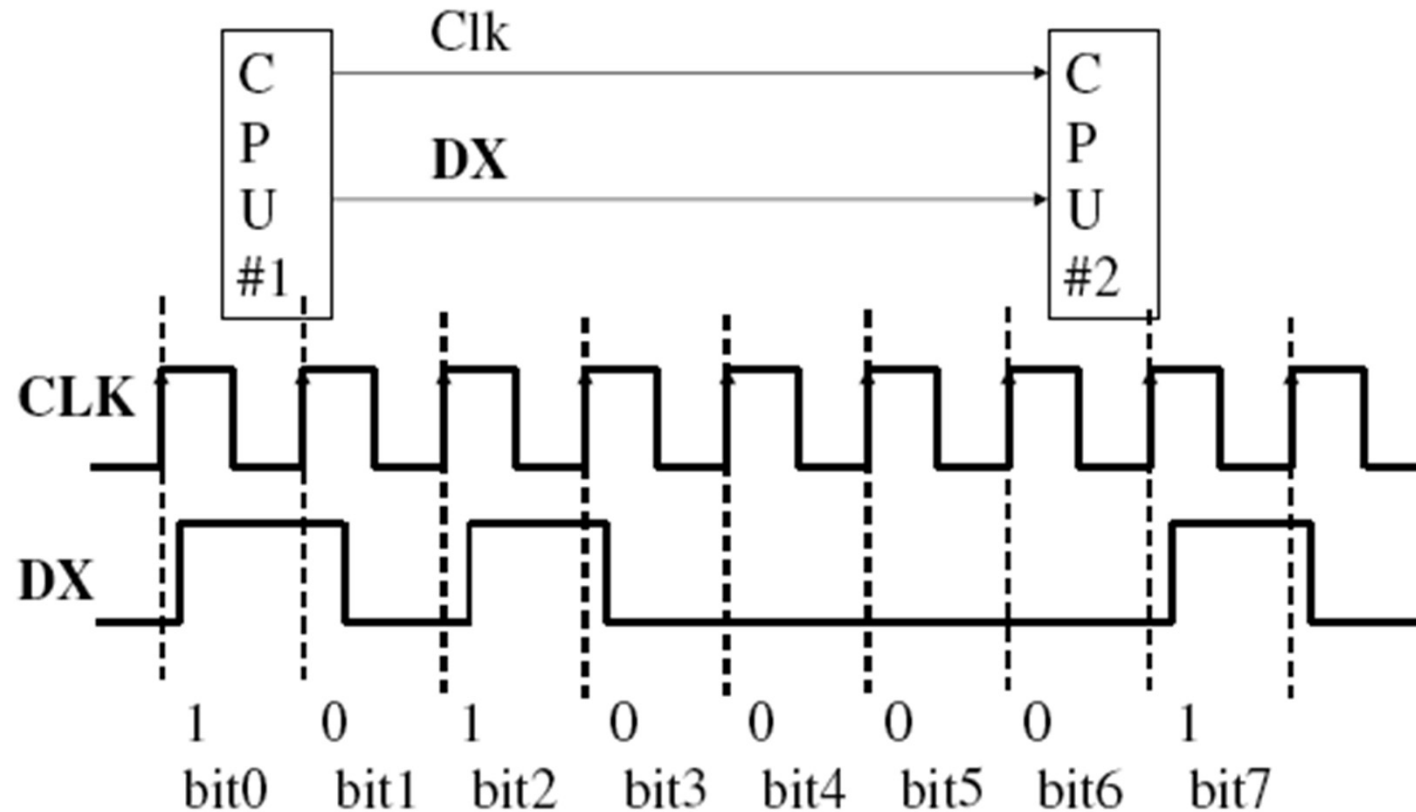
Application – parallel data transfer

- Multiple number of bits are transferred from one place to another (e.g. from one processor to the other processor)
- Use parallel registers in the source and destination
- Requires a lot of lines between the sources and destination, that may be expensive especially if the distance is long and the speed of transfer is high



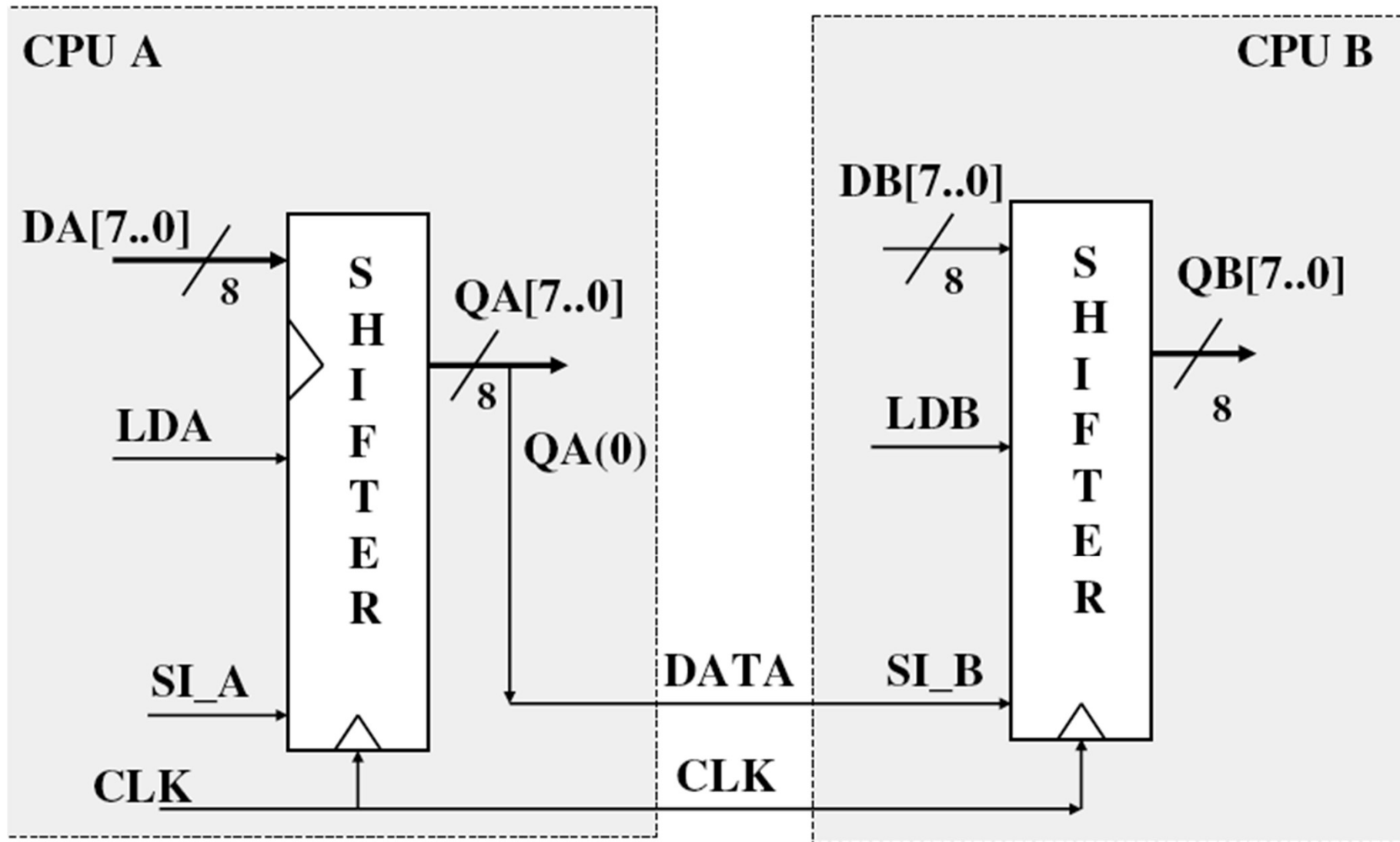
Application – serial data transfer

- To reduce the number of parallel wires, we can transfer data in serial fashion, i.e. one bit at a time.

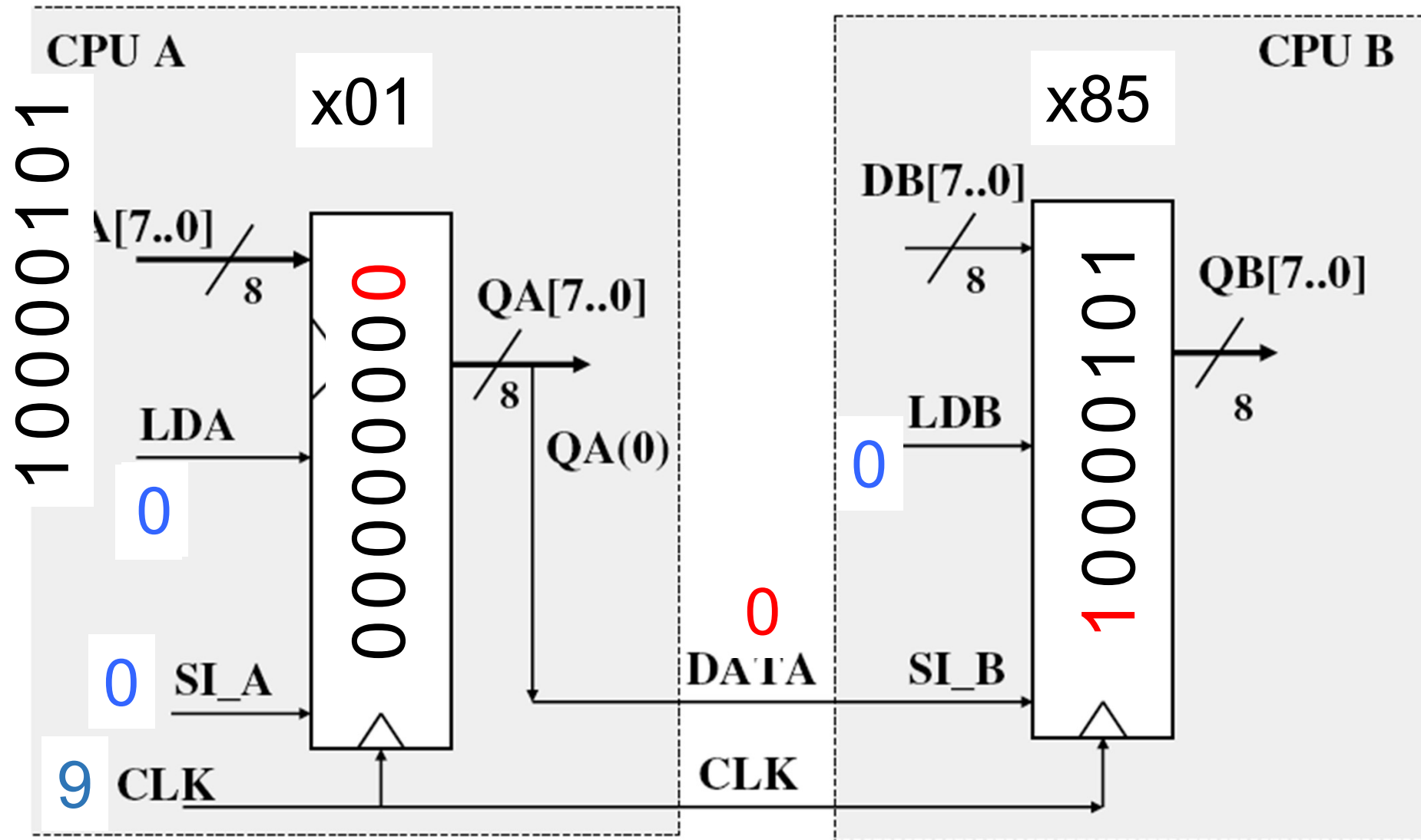


$x85 = 10000101$, here data is transmitted LSB to MSB

Application – serial data transfer

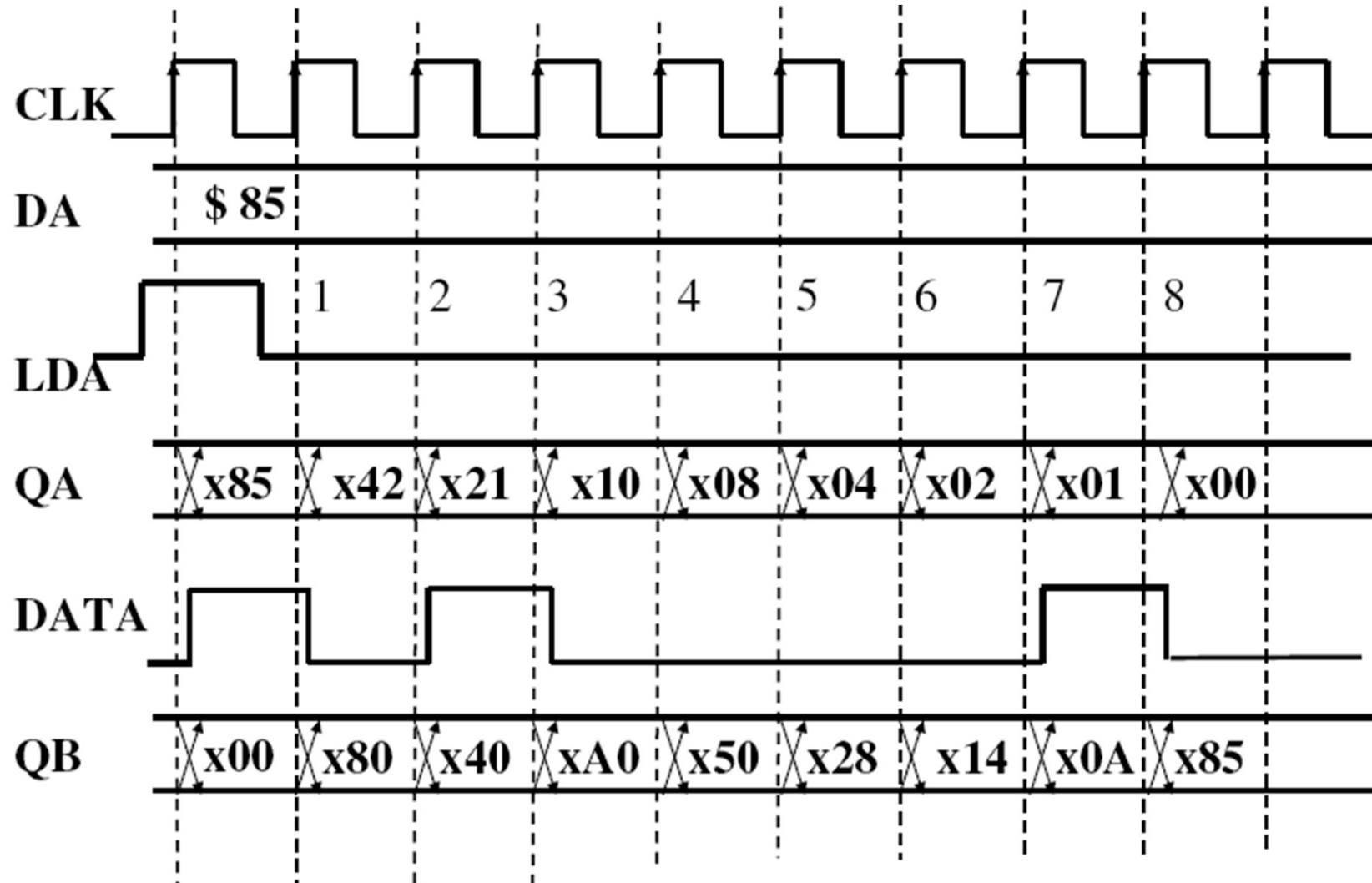


Application – serial data transfer



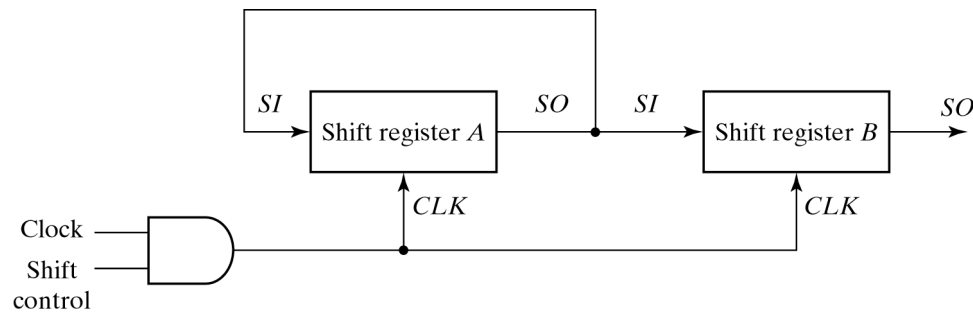
Application – serial data transfer

- $SI_A = 0$; QA, QB initially at x00

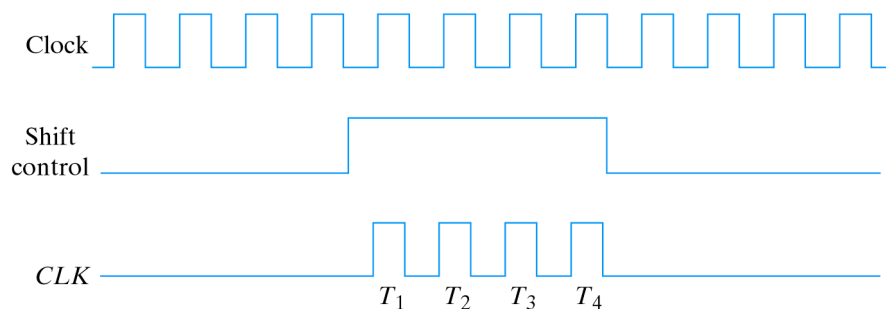


Application – serial data transfer

- A system is in serial mode when information is transferred one bit at a time
- E.g. we need to transfer a 4-bit data stored in a 4-bit register A to another 4-bit register B
- Use shift register to do serial transfer (i.e. transfer 1 bit at a time)
- In general, it takes n clock cycles to serially send n bits from source to destination



(a) Block diagram



(b) Timing diagram

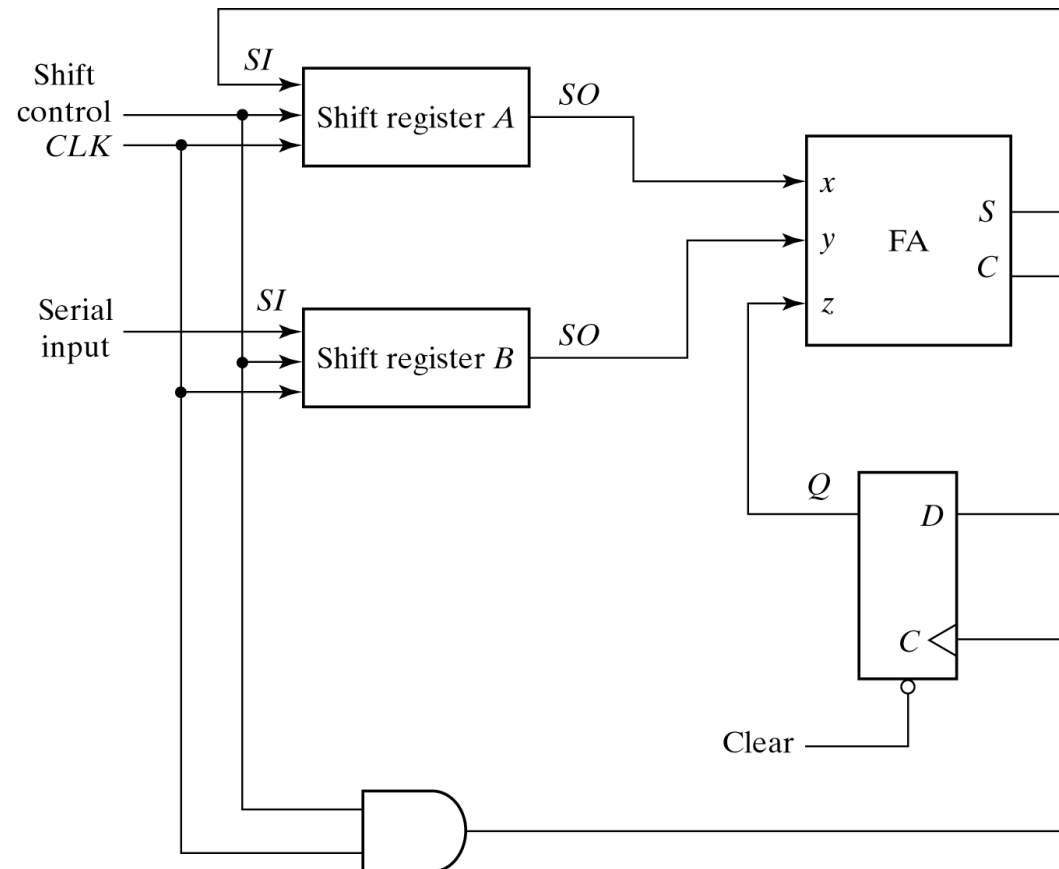
Timing pulse shift reg. A shift reg. B

Initial value	1	0	1	1	0	0	1	0
After T ₁	1	1	0	1	1	0	0	1
After T ₂	1	1	1	0	1	1	0	0
After T ₃	0	1	1	1	0	1	1	0
After T ₄	1	0	1	1	1	0	1	1

Data in A gets loaded into
B after 4 clock cycles

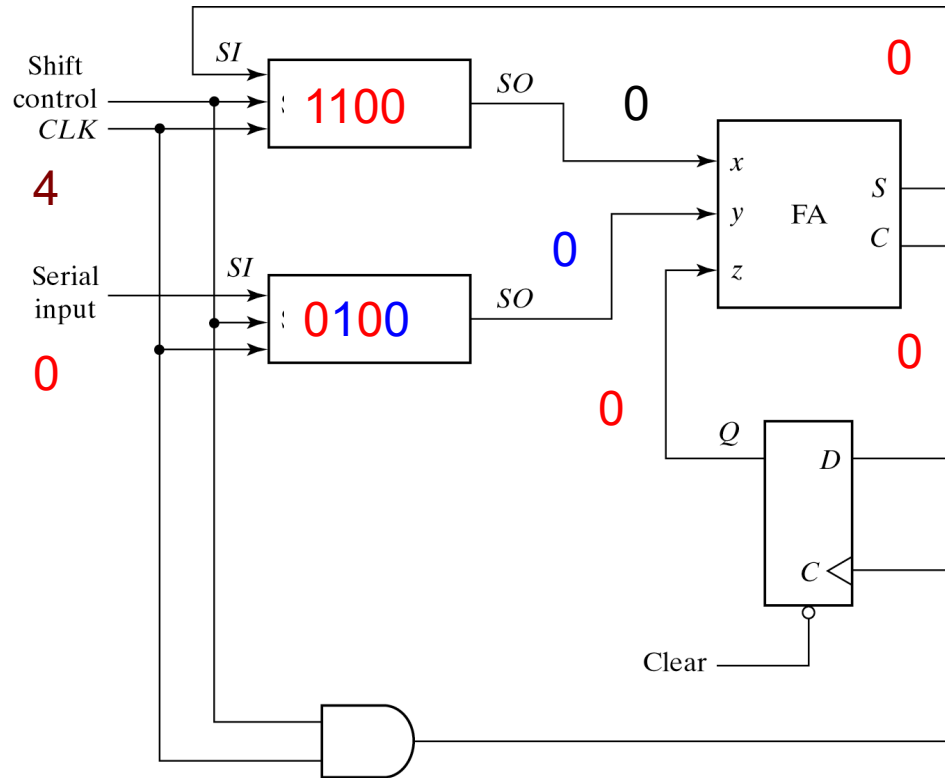
Application – serial adder

- Another usage for shift register is for serial computation.
- Normally we do computation in parallel, i.e. for n-bit addition, we do all the bit adding at the same time.
- Serial adder: use a 1-bit adder to do n-bit addition in multiple clock cycles

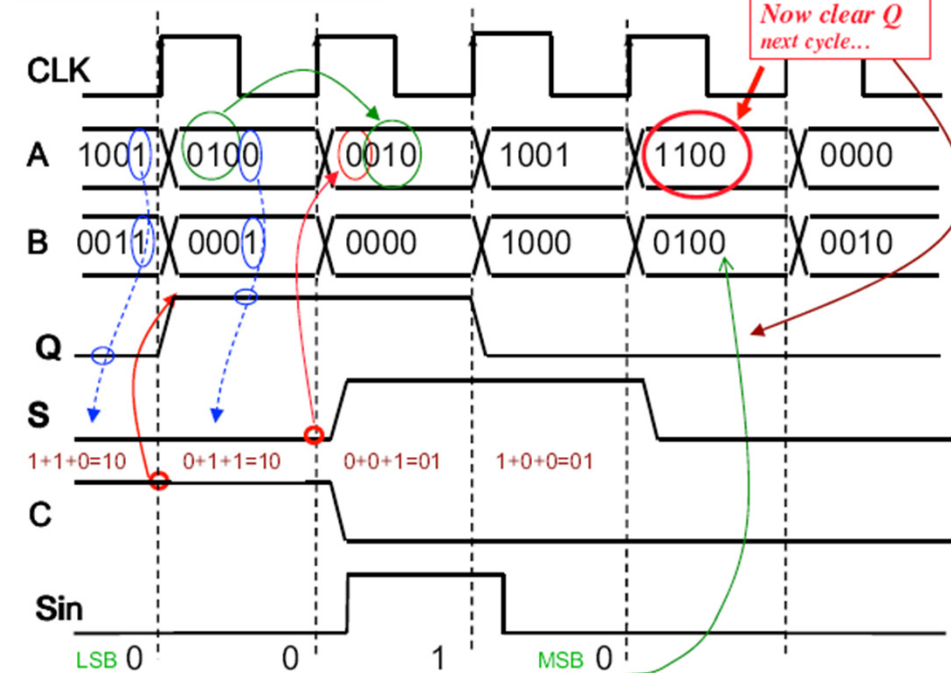


- Shift register A: the augend
- Shift register B: the addend
- Both the augend and addend are shifted to FA for addition
- The carry-input is through the DFF by a delay of one clock cycle

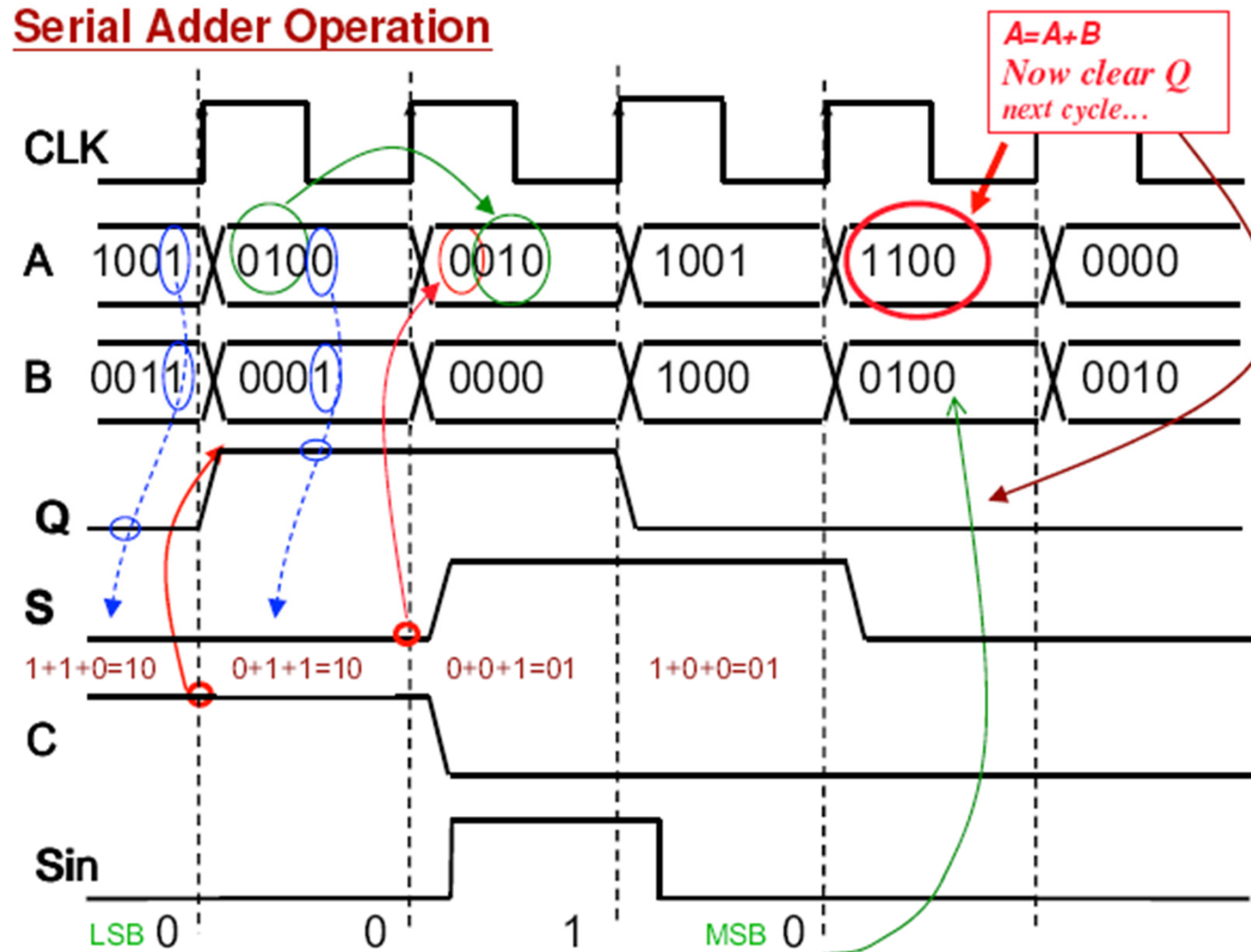
Application – serial adder



Serial Adder Operation



Application – serial adder





香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Thank you for listening !

